

Arquitectura de Integración

MODERNIZACION DE SISTEMAS BANCARIOS

JOSE LUIS RODRIGUEZ

Arquitectura de Integración para la Modernización de Sistemas Bancarios

Ejercicio Práctico de Arquitectura de Integración

Metodología: Top-Down de Arquitectura Empresarial (Flujos de Valor → Capacidades → Dominios → Aplicaciones/Tecnología)

1. Resumen Ejecutivo

Este documento presenta la propuesta de arquitectura de integración para la modernización de sistemas de un banco tradicional, cubriendo la integración de un Core Bancario tradicional con un nuevo Core Bancario digital, web banking y móvil banking, una plataforma de pagos, APIs de Open Finance, gestión de riesgos y prevención de fraude.

La propuesta contempla un enfoque top-down de Arquitectura Empresarial: partir de flujos de valor del negocio, se derivaron las capacidades necesarias, se agruparon en dominios (bounded contexts) validados contra el modelo de referencia BIAN, se definió la matriz de sistemas de registro, y finalmente se materializó en diagramas C4 (Contexto, Contenedores y Componentes).

2. Alcance y Escenario

Un banco tradicional busca modernizar su infraestructura tecnológica para mejorar sus servicios digitales y cumplir con nuevas tendencias de industria y regulaciones. Se requiere integrar:

- Core Bancario tradicional existente, más un nuevo Core Bancario digital.
- Nuevo sistema de banca web y banca móvil.
- Plataforma de servicios de pago.
- APIs para servicios de terceros (Open Finance).
- Sistema de gestión de riesgos.
- Sistema de prevención de fraudes.

3. Metodología: Enfoque Top-Down de Arquitectura Empresarial

El diseño parte de los flujos de valor (value streams): la secuencia de actividades que entregan valor a un stakeholder, desde un disparador hasta la realización del valor. De ahí se desciende a capacidades de negocio, luego a dominios/bounded contexts, y al final a la arquitectura de aplicaciones y tecnología.

- **1. Flujos de valor:** Qué valor se entrega y a quién.
- **2. Capacidades de negocio:** Qué debe poder hacer el negocio.
- **3. Dominios / bounded contexts:** Cómo se agrupan las capacidades (validado con BIAN).
- **4. Aplicaciones y tecnología:** Sistemas, APIs, eventos (diagramas C4).

4. Flujos de Valor

Se identificaron tres flujos principales:

#	Flujo de valor	Disparador	Valor entregado
1	Productos y cuentas bancarias	Cliente (directo o vía tercero autorizado) solicita/consulta un producto	Producto activo o información entregada
2	Banca digital (self-service)	Cliente inicia sesión en web/móvil	Consulta o transacción resuelta sin intervención humana
3	Procesamiento de pagos	Cliente (directo o vía tercero autorizado) origina una orden de pago	Fondos movidos, liquidados y confirmados

4.1 Etapas de los flujos

Flujo 1: Productos y cuentas bancarias

Solicitud/consulta (canal propio o tercero autorizado vía API) → autenticación/autorización → enrutamiento al sistema de registro (core tradicional vs. digital) → validación de elegibilidad y reglas de negocio (KYC, riesgo) → ejecución en el sistema de registro → publicación de eventos o captura de cambios → actualización de la vista agregada → confirmación y notificación.

Nota: este flujo no sincroniza los cores entre sí en la operación habitual: cada operación se ejecuta una sola vez, en el core propietario, y se propaga por eventos/CDC hacia la Vista Agregada (sección 7.1, 10). Si existe transferencia real de datos entre cores, ocurre únicamente durante el corte de una migración (sección 16.1), no como parte de este flujo.

Flujo 2: Banca digital (self-service)

Acceso al canal → autenticación (incl. MFA/step-up) → selección de operación → orquestación hacia backend (BFF/API Gateway) → agregación de datos multi-sistema → screening antifraude en operaciones sensibles → presentación del resultado → registro de auditoría.

Flujo 3: Procesamiento de pagos

Iniciación de la orden (canal propio o tercero autorizado vía API) → consentimiento y autenticación fuerte (SCA) → validación previa (fondos, límites, sanciones/PEP) → screening antifraude en tiempo real → enrutamiento al riel de pago → compensación y liquidación → confirmación y notificación → conciliación contable.

5. Mapa de Capacidades de Negocio

Flujo	Capacidad (L1)	Sub-capacidades (L2)	Service Domain(s) BIAN
F1	Originación y administración de productos bancarios	Originar producto	Customer Offer, Product Deployment
		Administrar cuenta/préstamo	Current Account, Savings Account, Consumer/Mortgage Loan
		Verificar identidad y cumplimiento	Party Authentication/Authorization, KYC
F2	Experiencia de banca digital	Gestionar permisos y restricciones de acceso al canal	Customer Access Entitlement
		Autenticar cliente (multi-nivel)	Party Authentication
		Mantener sesión técnica de interacción	Sin Service Domain BIAN*
		Registrar y analizar actividad del canal	Channel Activity Analysis, Customer Event History
F3	Procesamiento de pagos	Iniciar orden de pago	Payment Order Initiation
		Validar cumplimiento del pago	Sanction Screening
		Ejecutar y enrutar pago	Payment Execution
		Compensar y liquidar / Conciliar	Clearing and Settlement, Reconciliation
		Confirmar y notificar	Payment Order
Compartida	Gestión de riesgo	Evaluar riesgo crediticio / operacional	Credit Risk Rating, Operational Risk
Compartida	Prevención de fraude	Detectar / resolver fraude	Fraud Detection, Fraud Resolution
Habilitadora**	Observabilidad y trazabilidad	Monitoreo de disponibilidad, logging técnico	IT Service Availability
Habilitadora**	Integración e interoperabilidad	Exponer APIs y enrutar solicitudes	Sin equivalencia directa asignada
Habilitadora**	Integración e interoperabilidad	Adaptar protocolos y transformar mensajes	Sin equivalencia directa asignada

* La sesión técnica (token JWT, su emisión y renovación) no es una capacidad de negocio BIAN; es un detalle de implementación de IAM/canal, documentado en las secciones 8.2 y 11.

** Capacidades habilitadoras: no son exclusivas de un flujo, se consumen transversalmente desde los tres flujos principales.

Nota de mapeo: Customer Access Entitlement cubre permisos y restricciones de uso de producto y canal (qué puede hacer el cliente), no la autenticación (Party Authentication) ni la sesión técnica de interacción: se separan como tres sub-capacidades distintas. Channel Activity Analysis corresponde específicamente a la sub-capacidad de registro/análítica de comportamiento del canal (su alcance oficial: análisis de actividad y patrones de uso), distinta del control de acceso.

6. Dominios y Bounded Contexts

Se aplican principios DDD (responsabilidad única, ownership claro de datos, sin bases compartidas) para derivar los dominios a partir de las capacidades validadas.

Dominio	Datos bajo responsabilidad	Clasificación DDD	Interacción principal
Gestión de identidad y de personas y organizaciones	Perfil, credenciales, consentimientos	Generic	Síncrono (API)
Originación y administración de productos bancarios	Cuenta, préstamo, oferta	Core	Síncrono (API) + eventos de estado
Cumplimiento y KYC	Expediente KYC, resultado de verificación	Supporting	Síncrono (API, bloqueante en originación)
Riesgo	Score de riesgo, apetito/límites	Core	Síncrono (camino crítico)
Fraude	Score de fraude, caso de investigación	Core	Síncrono baja latencia + evento
Pagos	Orden de pago, transacción, liquidación	Core	Evento + síncrono puntual
Gestión de interacciones y canales digitales	Historial de interacción, persistido en la Plataforma de Observabilidad (sección 7); la sesión es efímera (JWT), no persistida por este dominio	Supporting	Síncrono (API) para la operación; evento para registro de historial

Nota: Integración/Orquestación y Observabilidad NO son dominios, sino componentes de arquitectura transversales sin datos de negocio propios. La excepción es el Historial de Interacción: la Plataforma de Observabilidad lo aloja físicamente (sección 7), pero su propietario funcional sigue siendo el dominio de Gestión de interacciones y canales digitales; alojar el dato no transfiere su propiedad de negocio. El API Gateway ejecuta enrutamiento técnico; no coordina procesos de negocio multi-dominio: esa orquestación (p. ej., el esquema de pago) vive dentro del dominio propietario del proceso (Payment Hub), no en el Gateway.

7. Matriz de Sistemas de Registro

Dominio	Sistema de Registro	Tipo	Rol
Originación y administración de productos bancarios	Core Bancario Tradicional (legado)	Legacy monolito	Autoritativo para productos no migrados
Originación y administración de productos bancarios	Core Bancario Digital (nuevo)	Nuevo / vendor	Autoritativo para productos nuevos y migrados bajo su propiedad
Cumplimiento y KYC	Plataforma KYC	COTS / utility	Expediente KYC y gestión de derechos del titular (no consentimientos, que permanecen en IAM; ver 11 y 12.1)
Riesgo	Motor de Riesgo	Nuevo / vendor scoring	Score de riesgo y límites
Fraude	Motor de Detección de Fraude	Nuevo / vendor ML	Score de fraude y casos
Pagos	Plataforma de Pagos (Payment Hub)	Nuevo / vendor payment hub	Orden, liquidación, conciliación
Gestión de identidad y de personas y organizaciones	Plataforma IAM	COTS / SaaS	Identidad, credenciales, consentimientos y perfil de la parte (única fuente autoritativa)

Dominio	Sistema de Registro	Tipo	Rol
Gestión de interacciones y canales digitales	Plataforma de Banca Digital (BFF)	Nuevo específico	Canal de interacción; sesión efímera (JWT), sin persistencia propia; publica eventos de interacción, no los almacena
Gestión de interacciones y canales digitales	Historial de Interacción (repositorio lógico alojado en la Plataforma de Observabilidad)	Lógico, sobre COTS/SaaS	Repositorio persistente y autoritativo del historial de interacción; alimentado por eventos publicados por el BFF vía Event Bus (ver fila de Observabilidad y 8.2)
Integración (no dominio)	API Gateway / iPaaS	Infraestructura	Enruta y compone, sin datos propios
Observabilidad (no dominio)	Plataforma de Observabilidad	COTS/SaaS	Telemetría técnica de toda la plataforma; también persiste el Historial de Interacción del canal (dominio Gestión de interacciones), alimentado por eventos del BFF vía Event Bus (ver 8.2)

7.1 Regla de propiedad del core dual

Cada producto tiene un único core propietario, responsable de sus datos y operaciones transaccionales. Los productos creados después de la fecha de corte del programa, o incorporados mediante una iniciativa digital-first expresamente habilitada, se originan en el Core Digital. Los productos existentes permanecen en el Core Tradicional hasta completar su migración. Acceder a un producto desde un canal digital no modifica su propiedad.

Registro de propiedad

El Registro de Propiedad de Cuentas mantiene la asignación de cada cuenta a su core propietario. Para otros productos, como préstamos, se aplica el mismo criterio mediante su identificador de producto. El registro conserva el propietario, una versión de asignación creciente y el estado operativo: activo o en migración.

El API Gateway consulta este registro para enrutar solicitudes, y el Ledger Adapter de Payment Hub lo consulta antes de ejecutar movimientos. Ambos utilizan una API de consulta común y no mantienen asignaciones autoritativas independientes.

El equipo responsable de la plataforma de migración administra el registro mediante operaciones autenticadas, autorizadas y auditadas. Su API administrativa permite registrar productos nuevos y modificar la propiedad durante migraciones o reversiones controladas.

Validación de escritura en los cores

Cada core conserva localmente la versión de propiedad y la habilitación de escritura del producto. Una operación solo se acepta cuando el core está habilitado como propietario y la versión recibida coincide con su versión vigente. Las solicitudes con versiones desactualizadas se rechazan antes de aplicar efectos.

Actualizar el registro central no deshabilita automáticamente al propietario anterior. Por ello, el cambio de propiedad requiere coordinar explícitamente el bloqueo del origen, la actualización del registro y la habilitación del destino. La seguridad del enrutamiento depende de este procedimiento y de la validación local, no únicamente de la tecnología utilizada para almacenar el registro.

Cambio de propietario

La migración mantiene como condición obligatoria que nunca existan dos cores habilitados simultáneamente para nuevas escrituras sobre el mismo producto.

Antes de transferir la propiedad, se impide iniciar nuevas operaciones y se completan o compensan las que ya estaban en curso. Durante esta etapa, únicamente se permiten las acciones controladas necesarias para cerrar esas operaciones. Una vez resueltas, se confirma el bloqueo de escritura del core de origen y se transfiere y verifica la información en el destino, que permanece bloqueado durante la preparación.

Con el origen bloqueado y la información validada, se actualiza la asignación con una nueva versión y se habilita el core de destino. El producto vuelve al estado activo después de verificar que el registro y las condiciones locales de ambos cores son coherentes. El procedimiento detallado se desarrolla en la sección 16.

Recuperación de una migración interrumpida

La herramienta de migración conserva un registro durable del proceso: identificador de migración, producto, origen, destino, versiones y acciones confirmadas. Las actualizaciones del registro central y de los cores se consideran operaciones separadas; no se presupone una transacción atómica entre los tres sistemas.

Si ocurre un fallo, se suspende la reapertura de operaciones y se comprueba el estado real de cada sistema. La recuperación utiliza el registro del proceso, las versiones y las habilitaciones locales para completar o revertir las acciones pendientes. Un indicador de escritura, por sí solo, no determina la propiedad.

La reversión sigue los mismos controles. Si el destino ya procesó movimientos, debe reconciliarse la información antes de devolver la propiedad al origen, conforme a la sección 16.2.

Disponibilidad y recuperación del registro

Si no puede consultarse una asignación válida, las operaciones que requieren identificar al core propietario fallan cerrado: no se utiliza un propietario por defecto ni se ejecutan escrituras basadas en una asignación incierta. Se aplican reintentos limitados y, si el fallo persiste, se escala como incidente.

El registro tiene una estrategia propia de HA/DR, descrita en la sección 15. Después de una restauración, las asignaciones afectadas se verifican contra las versiones locales y los registros de migración antes de reanudar el enrutamiento.

Consultas y agregación multicore

Las escrituras se ejecutan exclusivamente en el core propietario. Para presentar información consolidada, la plataforma utiliza composición de APIs de consulta o la Vista Agregada de Cliente, alimentada mediante eventos y captura de cambios de ambos cores.

La composición en el API Gateway se limita a combinar respuestas de consulta; las decisiones y procesos de negocio permanecen en los dominios responsables. La Vista Agregada admite consistencia eventual y no se utiliza para autorizar movimientos, validar fondos disponibles ni sustituir al core propietario en decisiones transaccionales.

8. Arquitectura de Alto Nivel: Diagramas C4

8.1 Diagrama de Contexto (C4, Nivel 1)

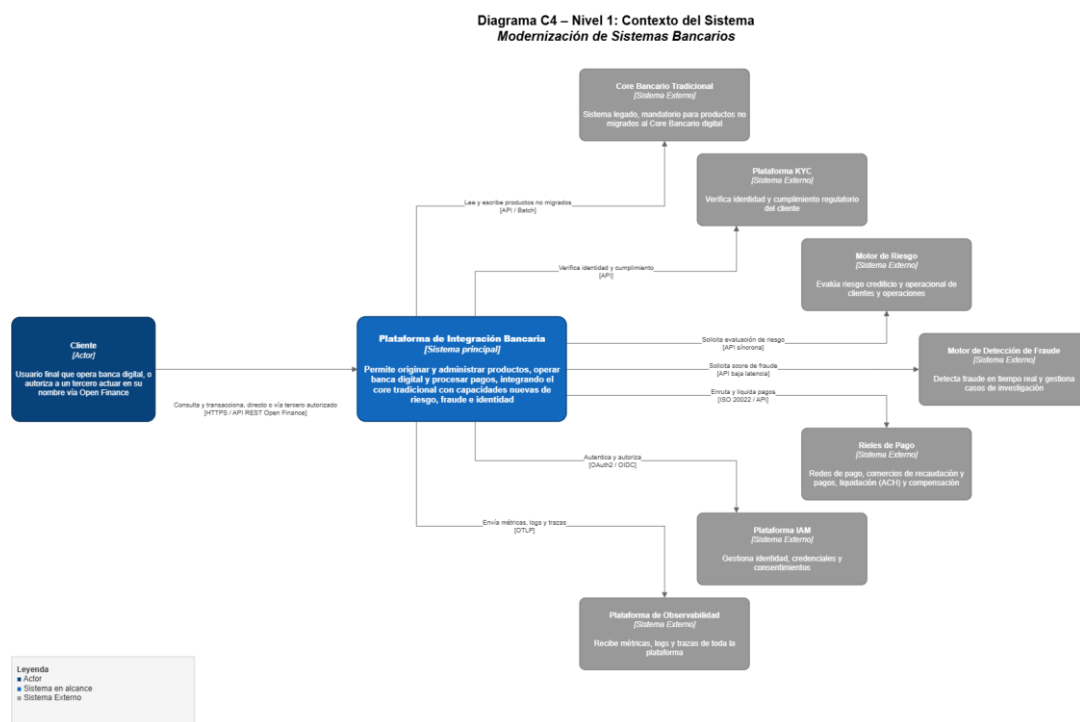


Figura 1. Diagrama C4 de Contexto: Plataforma de Integración Bancaria.

El Core Bancario Tradicional se modela como sistema externo (no como parte del sistema en alcance), reflejando que en el patrón Strangler Fig es un sistema existente que se integra, no que se rediseña.

Los diagramas 8.2 y 8.3, por su densidad de relaciones, se presentan en las páginas siguientes en formato horizontal para mayor legibilidad.

8.2 Diagrama de Contenedores (C4, Nivel 2)

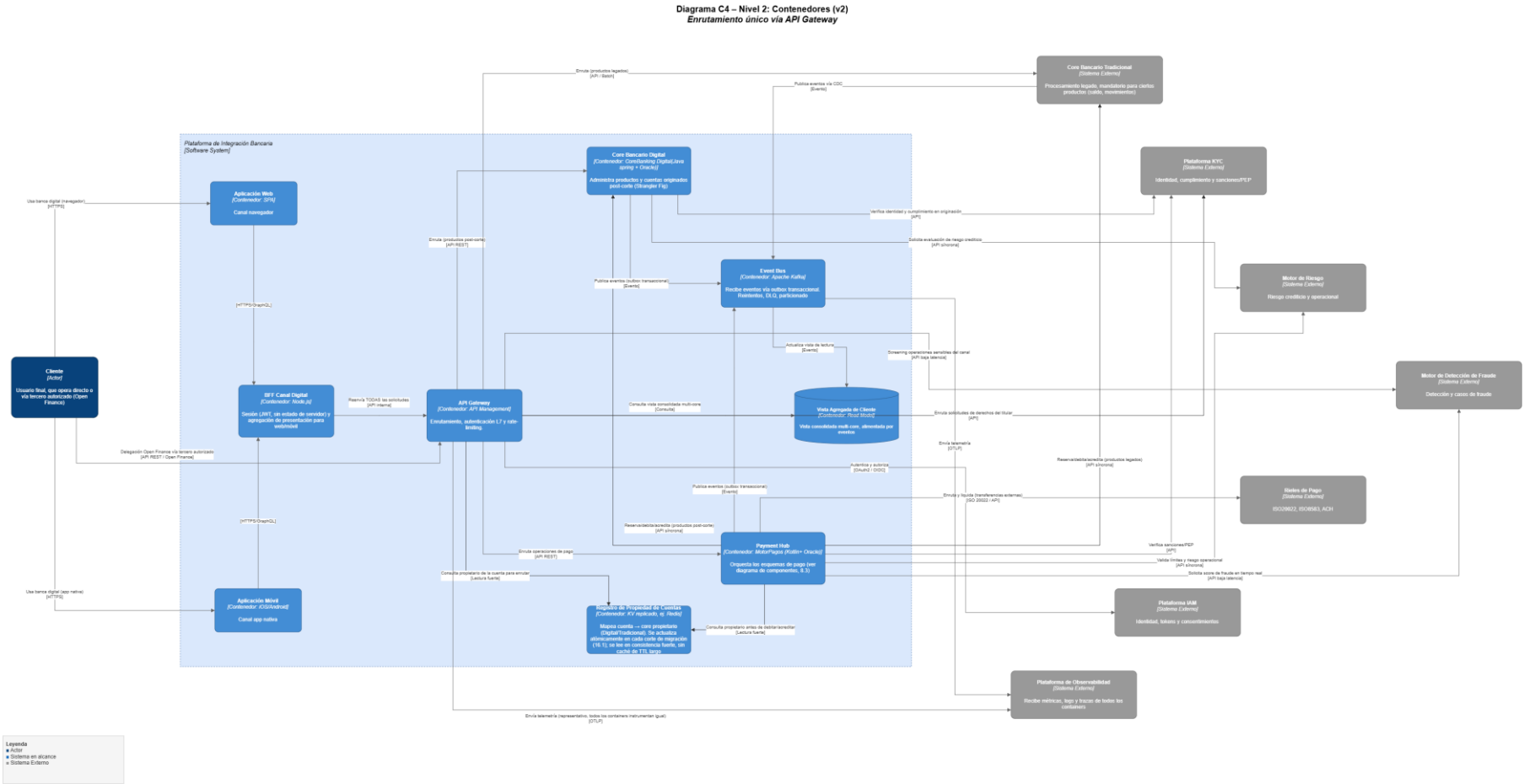


Figura 2. Diagrama C4 de Contenedores: enrutamiento único desde los canales hacia el Gateway; llamadas dominio-a-dominio y dominio-a-externo directas donde corresponde.

Container	Rol
Aplicación Web / Aplicación Móvil	Canales de cliente (SPA y app nativa), dentro del alcance de la plataforma; consumen el BFF
BFF Canal Digital	Sesión (JWT, sin estado de servidor) y agregación de presentación
API Gateway	Enrutamiento técnico, authN/Z L7 y rate-limiting para todo lo que entra desde los canales. No ejecuta lógica de negocio ni orquesta procesos multi-dominio
Core Bancario Digital	Productos/cuentas nuevos y migrados bajo su propiedad (Strangler Fig)
Payment Hub	Orquesta el esquema de pago (ver diagrama de componentes, 8.3)

Container	Rol
Registro de Propiedad de Cuentas	KV replicado (ej. Redis); mapea cuenta → core propietario con número de versión, validado por cada core al aceptar una escritura (sección 7.1)
Event Bus	Recibe eventos vía outbox transaccional; reintentos, DLQ, particionado
Vista Agregada de Cliente	Read model multi-core, evita bases de datos compartidas

La Plataforma de Observabilidad (externa) recibe telemetría de todos los contenedores; se muestran dos relaciones representativas en el diagrama (Gateway y Event Bus) para no saturarlo, pero la instrumentación es igual en todos. Además de telemetría técnica, esta plataforma persiste el Historial de Interacción del canal (dominio Gestión de interacciones, capítulo 6–7): el BFF publica eventos de actividad al Event Bus sin almacenarlos él mismo, y la Plataforma de Observabilidad los consume y conserva como parte de su función de logging/registro, distinta de las métricas puramente técnicas.

Alcance de "enrutamiento único": aplica a la entrada de tráfico desde los canales (Web, Móvil, BFF) y a la delegación Open Finance de un tercero autorizado: todo eso pasa exclusivamente por el API Gateway. No aplica a las llamadas dominio-a-dominio o dominio-a-sistema-externo (p. ej., Payment Hub → Core Digital para debitar, o Core Digital → Motor de Riesgo): esas son llamadas directas entre el propietario del proceso y su dependencia, un patrón east-west normal en microservicios, distinto del ingreso north-south que sí centraliza el Gateway.

Débito/crédito real: Payment Hub reserva, debita o acredita contra el core que sea propietario de la cuenta (Digital o Tradicional, según la regla de propiedad de la sección 7.1), llamando directamente a la API de ese core; nunca escribe en su base de datos, y esa llamada no pasa por el API Gateway porque no es tráfico de ingreso desde un canal.

8.3 Diagrama de Componentes: Payment Hub (C4, Nivel 3)

Se detalla a nivel de componentes el contenedor más crítico del sistema (Payment Hub), incluyendo sus dependencias externas (otros contenedores y sistemas externos), ya que es donde vive la orquestación del esquema de pago y donde se resuelve el patrón de idempotencia/outbox exigido por el procesamiento de pagos. El resto de los contenedores se mantiene a nivel de diagrama de contenedores por alcance del ejercicio.

Diagrama C4 – Nivel 3: Componentes de Payment Hub
Incluye dependencias externas del contenedor

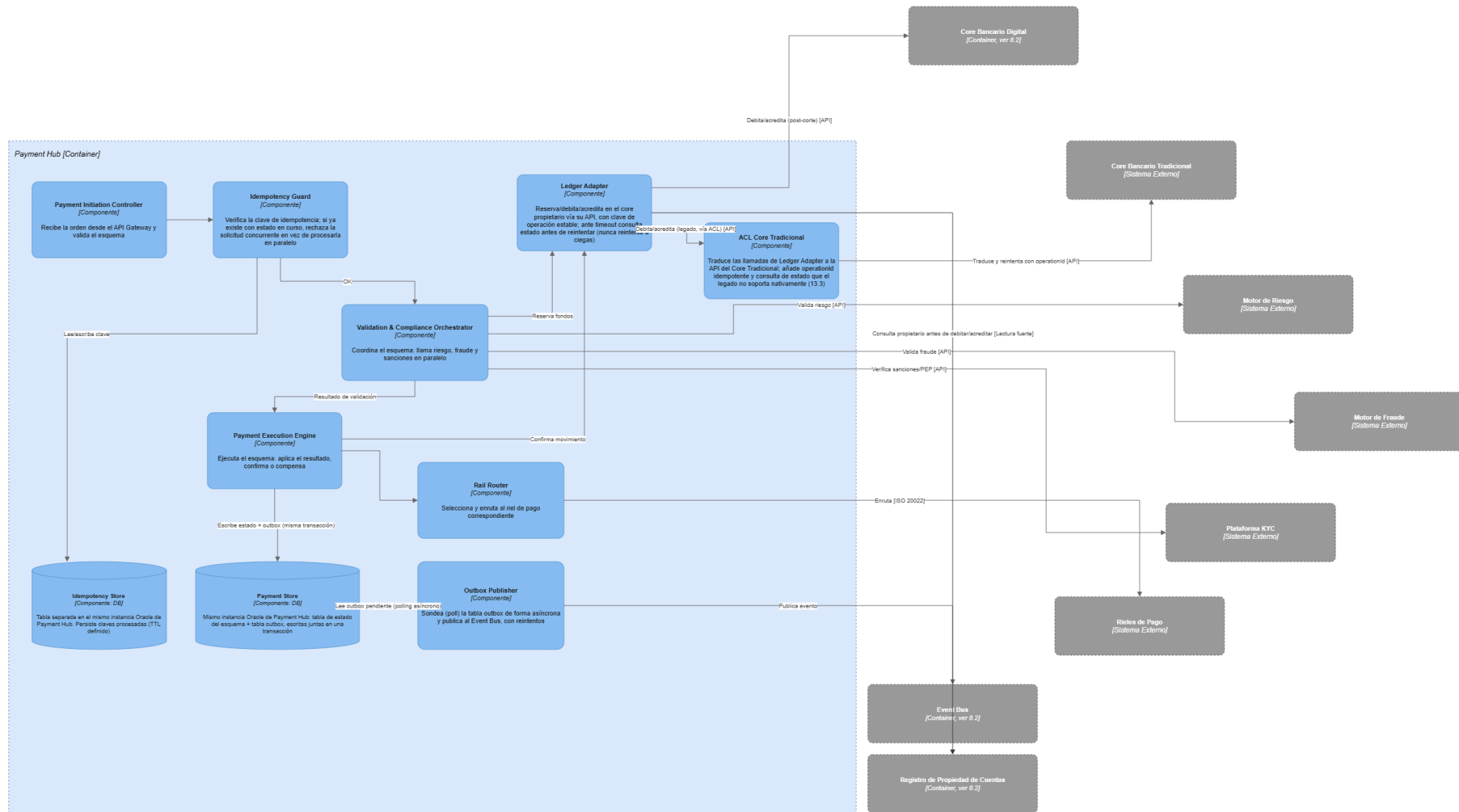


Figura 3. Diagrama C4 de Componentes: Payment Hub, con dependencias externas explícitas.

Cómo resuelve el problema de dual-write: Payment Execution Engine escribe el estado del pago y la fila outbox en la misma transacción de base de datos (ambas viven en el Payment Store, misma instancia Oracle de Payment Hub). Outbox Publisher, de forma asíncrona y separada, sondea (poll) esa tabla outbox y publica al Event Bus con reintentos: así, si la escritura del estado tiene éxito, el evento eventualmente se publica; y si la escritura falla, no hay evento huérfano. Idempotency Guard, respaldado por su propia tabla en la misma instancia Oracle, evita reprocesar la misma orden ante reintentos del cliente o del Gateway. El Payment Store se replica de forma síncrona hacia la región secundaria, aislado del resto del estado técnico de Payment Hub (15.1, 8.5); esta garantía cubre específicamente la identidad de la orden y sus operaciones, no el estado de ejecución reconstruible.

El Ledger Adapter, la Validation & Compliance Orchestrator y el Rail Router llaman directamente a sus dependencias externas (cores, motores de riesgo/fraude, KYC, rieles de pago); el diagrama las muestra explícitamente como referencias, ya detalladas en el diagrama de contenedores (8.2) o en el de contexto (8.1).

8.4 Diagramas C4 Dynamic: escenarios complejos

Los diagramas 8.1–8.3 muestran la estructura estática del sistema. Para los dos escenarios de mayor complejidad técnica del ejercicio (el corte de propiedad entre cores y el pago con fallback al legado), se detalla a continuación la secuencia real de interacciones (C4 Dynamic), numerada en el orden en que ocurre cada paso, para hacer explícitos el ordenamiento y las rutas de contingencia que la vista estática no puede representar.

Escenario 1: Corte de propiedad multicore

Corresponde al procedimiento de las secciones 7.1 y 16.1 cuando se decide transferir la propiedad de una cuenta entre el Core Bancario Tradicional y el Core Bancario Digital. No es una transacción atómica: la actualización del Registro de Propiedad de Cuentas y la habilitación de escritura local en el core destino son operaciones separadas, cada una registrada para permitir recuperación ante fallos (16.1, «Recuperación de un corte interrumpido»).

Diagrama C4 Dynamic — Corte de propiedad multicore

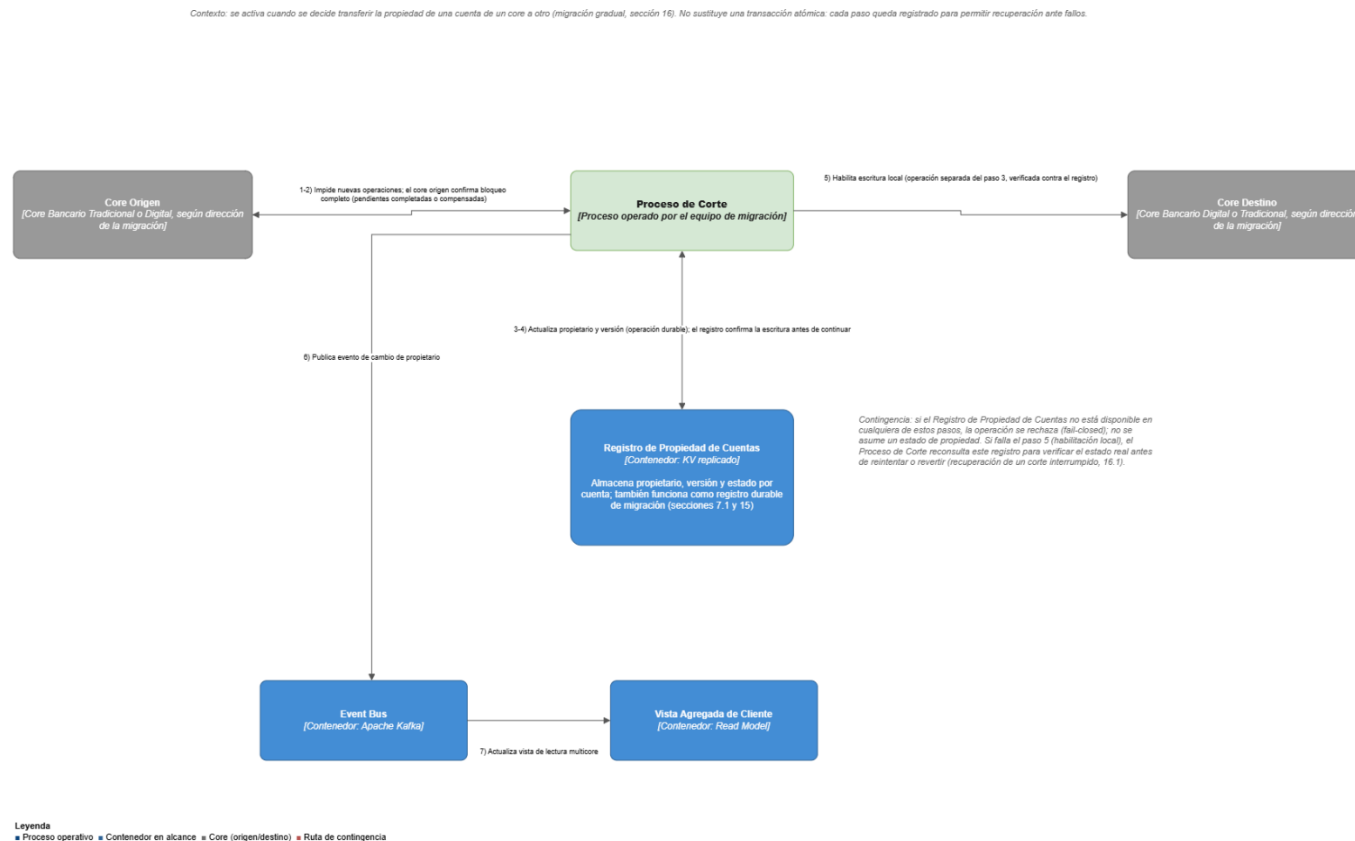


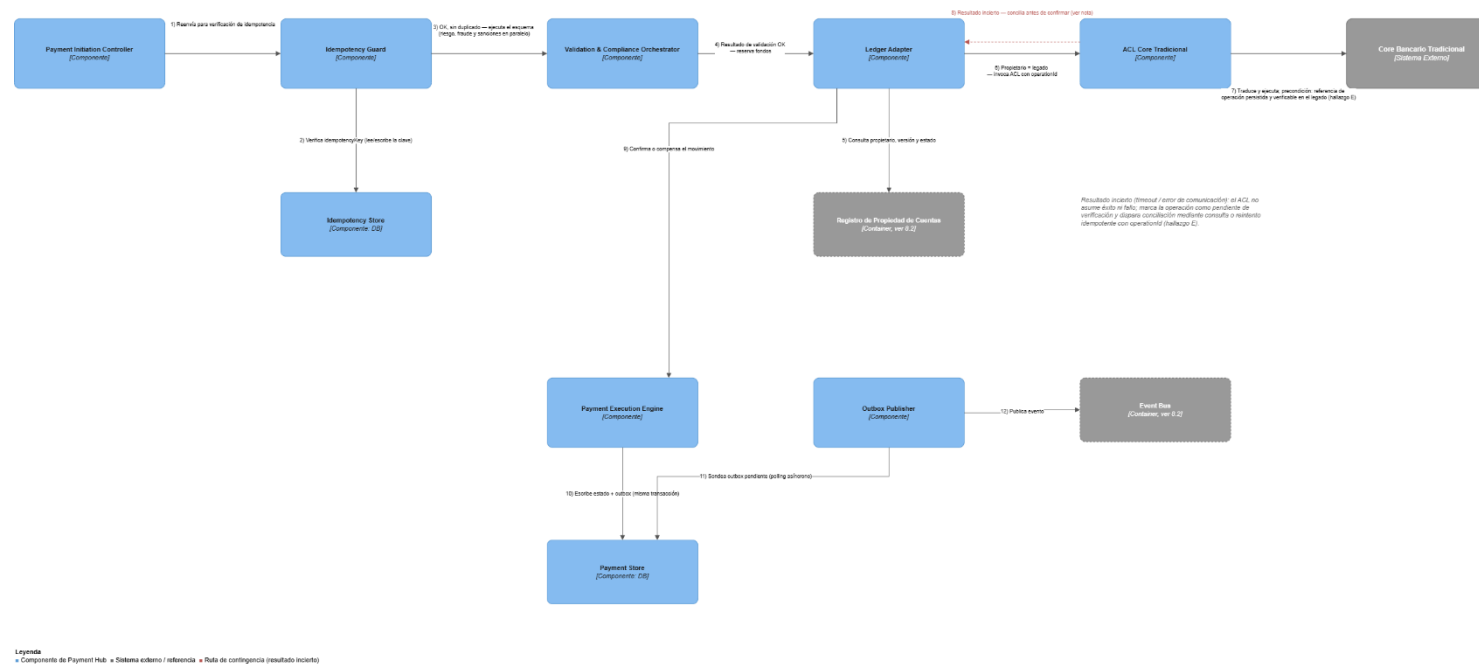
Figura 4. Diagrama C4 Dynamic: Corte de propiedad multicore.

- **Pasos 1–2:** el Proceso de Corte impide nuevas operaciones en el core origen; este confirma bloqueo completo (pendientes completadas o compensadas).
- **Pasos 3–4:** se actualiza propietario y versión en el Registro de Propiedad de Cuentas (operación durable); el registro confirma la escritura antes de continuar.
- **Paso 5:** se habilita la escritura local en el core destino (operación separada de los pasos 3–4, verificada contra el registro, no simultánea con ellos).

- **Pasos 6–7:** se publica el evento de cambio de propietario en el Event Bus, que actualiza la Vista Agregada de Cliente.
- **Contingencia:** si el registro no está disponible en cualquiera de estos pasos, la operación se rechaza (fail-closed). Si falla el paso 5, el Proceso de Corte reconsulta el registro para verificar el estado real por cuenta antes de reintentar o revertir, en vez de asumir un resultado.

Detalla, a nivel de componentes del Payment Hub, el caso en que la cuenta involucrada en una orden de pago pertenece al Core Bancario Tradicional (secciones 13.2 y 13.3). Explicita la precondition verificable que exige el ACL Core Tradicional y el tratamiento de un resultado incierto; ninguno de los dos queda resuelto por el outbox transaccional por sí solo.

Contexto: la orden ya fue enrutada por el API Gateway al Payment Hub (ver Nivel 2). Este diagrama detalla el caso en que la cuenta pertenece al core tradicional (legado).



- **Pasos 1–2:** el Payment Initiation Controller reenvía la orden al Idempotency Guard, que verifica la idempotencyKey contra el Idempotency Store; si ya existe una operación en curso, rechaza la solicitud concurrente en vez de procesarla en paralelo.
- **Pasos 3–4:** el Validation & Compliance Orchestrator ejecuta el esquema (riesgo, fraude y sanciones en paralelo); con resultado favorable, el Ledger Adapter reserva fondos.
- **Paso 5:** el Ledger Adapter consulta propietario, versión y estado en el Registro de Propiedad de Cuentas.
- **Pasos 6–7:** al ser el propietario el core legado, se invoca al ACL Core Tradicional, que traduce y ejecuta la operación con un operationId (bajo la precondition de que exista una referencia de operación persistida y verificable en el legado, no solo el envío de la solicitud).
- **Paso 8 (contingencia):** ante timeout o error de comunicación, el ACL no asume éxito ni fallo; marca la operación como pendiente de verificación y dispara conciliación mediante consulta o reintento idempotente con el mismo operationId.

- **Paso 9:** el Ledger Adapter confirma o compensa el movimiento en el Payment Execution Engine según el resultado verificado.
- **Pasos 10–12:** se escribe el estado y el outbox en la misma transacción (Payment Store); el Outbox Publisher sondea el outbox de forma asíncrona y publica el evento en el Event Bus.

8.5 Diagrama de Arquitectura en la Nube (Microsoft Azure)

Los diagramas anteriores son agnósticos de proveedor. Para demostrar viabilidad de despliegue concreta, esta sección mapea los contenedores del Nivel 2 a servicios administrados de Microsoft Azure, alineado con la estrategia de HA/DR del capítulo 15: multi-AZ dentro de la región primaria y una región secundaria en modo standby cálido con replicación asíncrona.

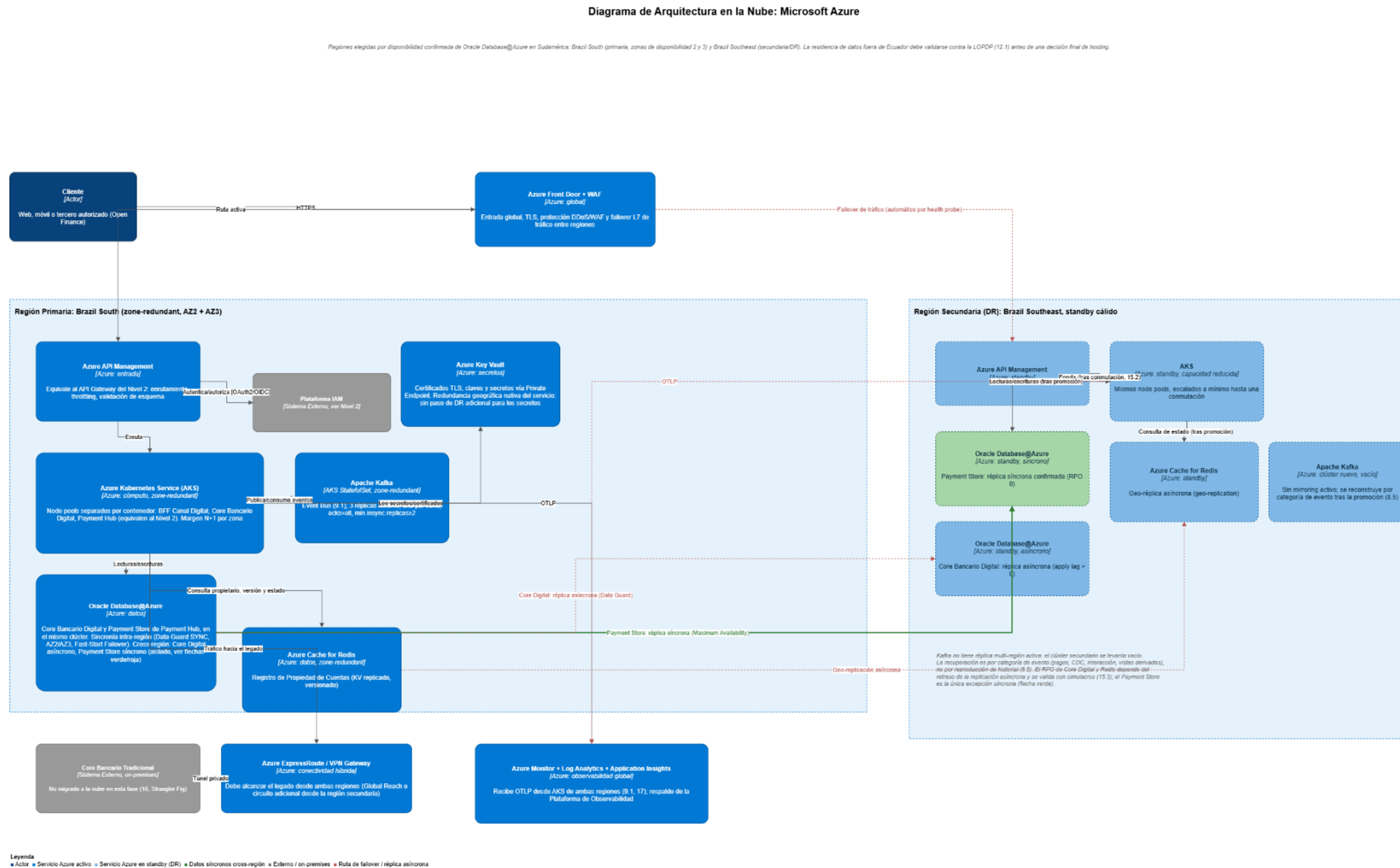


Figura 6. Diagrama de arquitectura en la nube: Microsoft Azure (regiones, redes y HA/DR).

- **Regiones:** Brazil South como región primaria, zone-redundant entre las zonas de disponibilidad 2 y 3; Brazil Southeast como región secundaria en standby cálido. Ambas confirmadas con soporte de Oracle Database@Azure (no todas las regiones de Azure lo ofrecen). Pendiente: validar la residencia de datos fuera de Ecuador contra la LOPDP (12.1) antes de tomar esta decisión como definitiva.
- **Entrada y cómputo:** Azure Front Door concentra el enrutamiento global, TLS y protección DDoS/WAF, con failover L7 hacia la región secundaria. Azure API Management cumple el rol del API Gateway del Nivel 2. Azure Kubernetes Service (AKS), zone-redundant, aloja BFF Canal Digital, Core Bancario Digital y Payment Hub en node pools separados; Apache Kafka corre como StatefulSet en el mismo clúster, con al menos una réplica por zona.
- **Datos:** Oracle Database@Azure sirve a Core Bancario Digital y Payment Hub, en sincronía (Data Guard/RAC) entre zonas de la región primaria. Azure Cache for Redis, también zone-redundant, aloja el Registro de Propiedad de Cuentas. Azure Key Vault gestiona certificados y secretos vía Private Endpoint. La redundancia geográfica de Key Vault es nativa del servicio, por lo que no requiere el mismo procedimiento de promoción que las bases de datos: los secretos están disponibles en la región secundaria sin un paso adicional de failover.
- **Recuperación ante desastres:** el resto de Oracle Database@Azure (Core Bancario Digital) y Azure Cache for Redis replican de forma asíncrona hacia la región secundaria; el RPO real depende del retraso de esa replicación y debe medirse con simulacros (15.3), no asumirse cercano a cero. El Payment Store de Payment Hub es la excepción: replica de forma síncrona (Data Guard, modo Maximum Availability), con Payment Hub en fail-closed sobre el registro de órdenes si esa confirmación se degrada (15.1). Kafka no cuenta con réplica multi-región activa en este diseño: la región secundaria levanta un clúster vacío, y la recuperación es por categoría de evento, no por replicación de historial. Los eventos de pago se republican desde el Payment Store, ya síncrono; los eventos CDC del legado se reanudan desde el propio core una vez reconectado, sujeto a la retención de su log de cambios; los eventos de interacción del canal (BFF) sin consumir antes de la pérdida de región se pierden, sin fuente alterna; y las vistas derivadas (Vista Agregada) se resincronizan por backfill contra cada sistema de registro propietario, no reproduciendo el historial de eventos (17).
- **Legado y observabilidad:** el Core Bancario Tradicional permanece on-premises en esta fase (16, Strangler Fig) y se conecta vía Azure ExpressRoute o VPN Gateway. Azure Monitor, Log Analytics y Application Insights reciben telemetría OTLP de ambas regiones, cumpliendo el rol de la Plataforma de Observabilidad (9.1, 17). La conectividad al legado debe alcanzar ambas regiones, vía Azure ExpressRoute Global Reach o un segundo circuito dedicado desde la región secundaria; sin eso, el legado queda inalcanzable tras un failover regional y el Escenario C (15.2) no puede aplicarse desde el sitio de recuperación.

9. Patrones de Integración y Tecnologías

La arquitectura combina APIs síncronas para operaciones que requieren respuesta inmediata y eventos para propagar cambios y actualizar información derivada. Los patrones se aplican según la responsabilidad de cada componente, manteniendo la propiedad de los datos en los dominios y la coordinación de procesos de negocio en sus servicios responsables.

Patrón / Tecnología	Dónde se aplica	Por qué
Backend for Frontend (BFF)	Atención de los canales web y móvil	Adapta solicitudes y respuestas a las necesidades de presentación. Las llamadas a APIs de negocio pasan por el Gateway. La publicación de eventos de interacción al Event Bus constituye una integración asíncrona separada
API Gateway (puro enrutamiento)	Único punto de entrada para tráfico de ingreso desde canales y Open Finance	Centraliza validación de identidad y permisos, límites de consumo y enrutamiento según el registro de propiedad. Las reglas y decisiones de negocio se ejecutan en los servicios responsables
API Composition	Consultas agregadas mediante el Gateway	Combina respuestas de varios sistemas para presentar información consolidada. Su alcance se limita a consultas; no coordina escrituras ni procesos de negocio distribuidos
CQRS / Read Model	Vista Agregada de Cliente	Desacopla lectura de escritura. Separa las consultas consolidadas de las operaciones transaccionales. La vista se alimenta por eventos o CDC y admite consistencia eventual; no se utiliza para validar fondos ni autorizar movimientos
Event-Driven Architecture	Backbone entre dominios (Event Bus)	Desacopla productores/consumidores; permite propagar cambios sin que el productor tenga que esperar a todos los consumidores
Transactional Outbox	Core Digital y Payment Hub	Registra el cambio de negocio y el evento pendiente dentro de una misma transacción local. Un publicador entrega posteriormente el evento al broker con reintentos. Los consumidores deben tolerar duplicados
Change Data Capture (CDC)	Core Tradicional → Event Bus	Integra el legado sin acoplarlo síncronamente, condicionado a que el legado exponga una fuente de captura de cambios viable (log de transacciones, triggers u otro mecanismo) y a los permisos que su administrador otorgue sobre ella. No se garantiza en general que la integración sea posible "sin modificarlo"; si el legado no ofrece esa capacidad, se requiere evaluar una alternativa (p. ej., extracción periódica) antes de comprometerse con este patrón
Strangler Fig	Migración Core Tradicional → Core Digital	Permite trasladar productos por etapas, manteniendo un único propietario y verificando cada corte antes de ampliar el alcance

Patrón / Tecnología	Dónde se aplica	Por qué
Circuit Breaker / Bulkhead	Llamadas a Riesgo, Fraude, KYC (externos)	Limitan la espera y aíslan fallos para evitar su propagación. Las verificaciones obligatorias de una operación financiera no se omiten por indisponibilidad de una dependencia
Anti-Corruption Layer	Componente ACL Core Tradicional dentro del Payment Hub	Traduce contratos y particularidades del legado para mantener al Ledger Adapter independiente de su implementación. La identificación y consulta de operaciones ejecutadas dependen de una referencia verificable que el legado pueda conservar y consultar
Esquema orquestado (patrón Saga)	Payment Hub (iniciación → validación → reserva/débito/crédito → confirmación)	Coordina transacciones distribuidas sin transacción global (2PC); la orquestación vive en el dominio propietario del proceso, no en el Gateway

Principios reflejados: cada patrón sostiene cohesión, bajo acoplamiento, responsabilidades claras y evolución independiente. Ningún microservicio comparte base de datos con otro; el API Gateway no contiene reglas de negocio ni orquesta procesos multi-dominio; los eventos llevan payload de referencia suficiente para no forzar consultas síncronas adicionales en el camino crítico.

9.1 Tecnologías propuestas

Las tecnologías indicadas constituyen una propuesta de implementación. Su adopción debe validarse contra las capacidades del equipo, licenciamiento, soporte y objetivos de disponibilidad y latencia del banco.

Tecnología	Aplicación	Motivo de selección
Plataforma de API Management	API Gateway y gestión de APIs	Reúne políticas de acceso, control de consumo, publicación y seguimiento del ciclo de vida de las APIs. La selección del producto depende de su compatibilidad con IAM, Open Finance y los requisitos operativos.
Node.js	BFF de los canales	Adecuado para coordinar solicitudes de entrada/salida y adaptar respuestas de presentación. Su uso se limita a la lógica del canal; las reglas transaccionales permanecen en los servicios de negocio.
Java con Spring	Servicios del Core Digital	Permite estructurar APIs, validaciones, acceso transaccional a datos y adaptadores de integración. Su selección debe corresponder al modelo de implementación del core y al soporte disponible.
Kotlin sobre JVM	Payment Hub	Permite implementar la coordinación de pagos utilizando el ecosistema JVM y sus bibliotecas. La coexistencia con Java requiere convenciones comunes de desarrollo, pruebas y operación.
Oracle	Persistencia transaccional del Core Digital y Payment Hub	Soporta las transacciones locales necesarias para mantener consistencia entre datos de negocio, registros de idempotencia y outbox. Cada servicio conserva almacenamiento propio y acceso exclusivo a sus datos. La elección debe justificar licenciamiento y operación.
Apache Kafka	Event Bus	Permite conservar y distribuir eventos, desacoplar consumidores y reprocesar información. El particionado, la retención y la replicación se dimensionan según volumen y recuperación; el orden se gestiona dentro de cada partición.

Tecnología	Aplicación	Motivo de selección
Almacén clave-valor replicado, por ejemplo Redis	Registro de Propiedad de Cuentas	Facilita la consulta de asignaciones por identificador. La elección debe satisfacer los requisitos de recuperación; la protección frente a escrituras en el core equivocado se completa con versiones y habilitación local, conforme a 7.1.
OpenTelemetry y plataforma de observabilidad	Métricas, logs y trazas	Proporcionan instrumentación y correlación entre componentes. Si la plataforma también conserva el historial de interacción, este se trata como un repositorio lógico con políticas propias de acceso, retención y recuperación.

GraphQL se utiliza en la interfaz entre los canales y el BFF, documentada mediante su propio esquema GraphQL. Las APIs de servicios y Open Finance utilizan REST sobre HTTPS y contratos JSON, documentados con OpenAPI. Los eventos se documentan con AsyncAPI y esquemas versionados, según la estrategia del capítulo 13.

10. Estrategia de Integración Multicore

Ver regla de propiedad y mecanismos de agregación en la sección 7.1. En síntesis: enrutamiento por regla de propiedad explícita + composición o read model para lectura, nunca sincronización bidireccional de las bases del core tradicional y el core digital.

- **Escritura:** cada operación de escritura va exclusivamente al core propietario del producto, determinado por la regla de corte.
- **Lectura agregada:** vía API Composition (tiempo real) o Vista Agregada de Cliente (CQRS, alimentada por eventos/CDC).
- **Consistencia:** eventual entre la Vista Agregada y los cores fuente; aceptable porque la vista es de solo lectura y no participa en decisiones transaccionales.

11. Gestión de Identidad y Acceso

- **Autenticación:** OAuth2/OIDC como estándar en todos los canales; MFA obligatorio para operaciones sensibles (pagos de alto monto, cambio de datos de contacto).
- **Autorización:** modelo basado en roles/scopes (RBAC) para operaciones internas; modelo de consentimiento granular por scopes para terceros Open Finance, alineado a estándares FAPI (Financial-grade API).
- **Gestión de tokens:** tokens de acceso de corta duración (JWT, 5–15 min) con refresh token; revocación centralizada en la Plataforma IAM.
- **Identidad de servicio:** mTLS entre contenedores internos (Core Digital, Payment Hub, Event Bus); cada dominio tiene su propia service account, sin credenciales compartidas.
- **Consentimiento Open Finance:** capturado, versionado y con expiración explícita en la Plataforma IAM; el API Gateway valida el scope del consentimiento en cada llamada delegada de un tercero autorizado.

11.1 Continuidad ante degradación del IAM

Cachear las claves públicas de verificación (JWKS) en el API Gateway solo demuestra que un token fue emitido y no ha sido alterado; no demuestra que la sesión o el consentimiento sigan vigentes en este momento (un token con firma válida puede corresponder a una sesión ya cerrada o a un consentimiento ya revocado). Por eso se distingue explícitamente:

- **Operaciones de baja sensibilidad (consultas sobre datos no sensibles):** pueden aceptar la verificación de firma vía JWKS cacheado durante una degradación breve del IAM. La política depende de la sensibilidad del dato y del permiso involucrado, no de si la operación es "de solo lectura": una lectura de datos sensibles ya expuesta no es reversible aunque la operación en sí no modifique nada. Por eso este nivel se reserva explícitamente para consultas de bajo impacto.
- **Operaciones de alta sensibilidad (pagos, consultas que exponen datos personales o financieros sensibles, y cualquier llamada que dependa de un consentimiento Open Finance):** exigen una verificación de vigencia en tiempo real contra la Plataforma IAM (estado de sesión y de consentimiento), no solo la firma. Si el IAM no

responde, el sistema falla cerrado (fail-closed) para estas operaciones (se rechazan, no se procesan en modo degradado), precisamente porque son las de mayor impacto ante una revocación no detectada.

- **Mitigación del tiempo de bloqueo:** una lista de revocación local, de vigencia corta y refrescada periódicamente desde el IAM, puede reducir la ventana de fail-closed, pero se trata siempre como información best-effort, no autoritativa. Nunca sustituye la verificación en tiempo real para pagos.

11.2 Diagrama de Componentes del Dominio IAM

El punto anterior describe las reglas de autenticación, autorización, tokens, identidad de servicio y consentimiento a nivel de decisión. La «Plataforma IAM» referenciada en el Nivel 2 (8.2) se detalla aquí como un sistema con cuatro componentes propios, cada uno responsable de una de esas reglas.

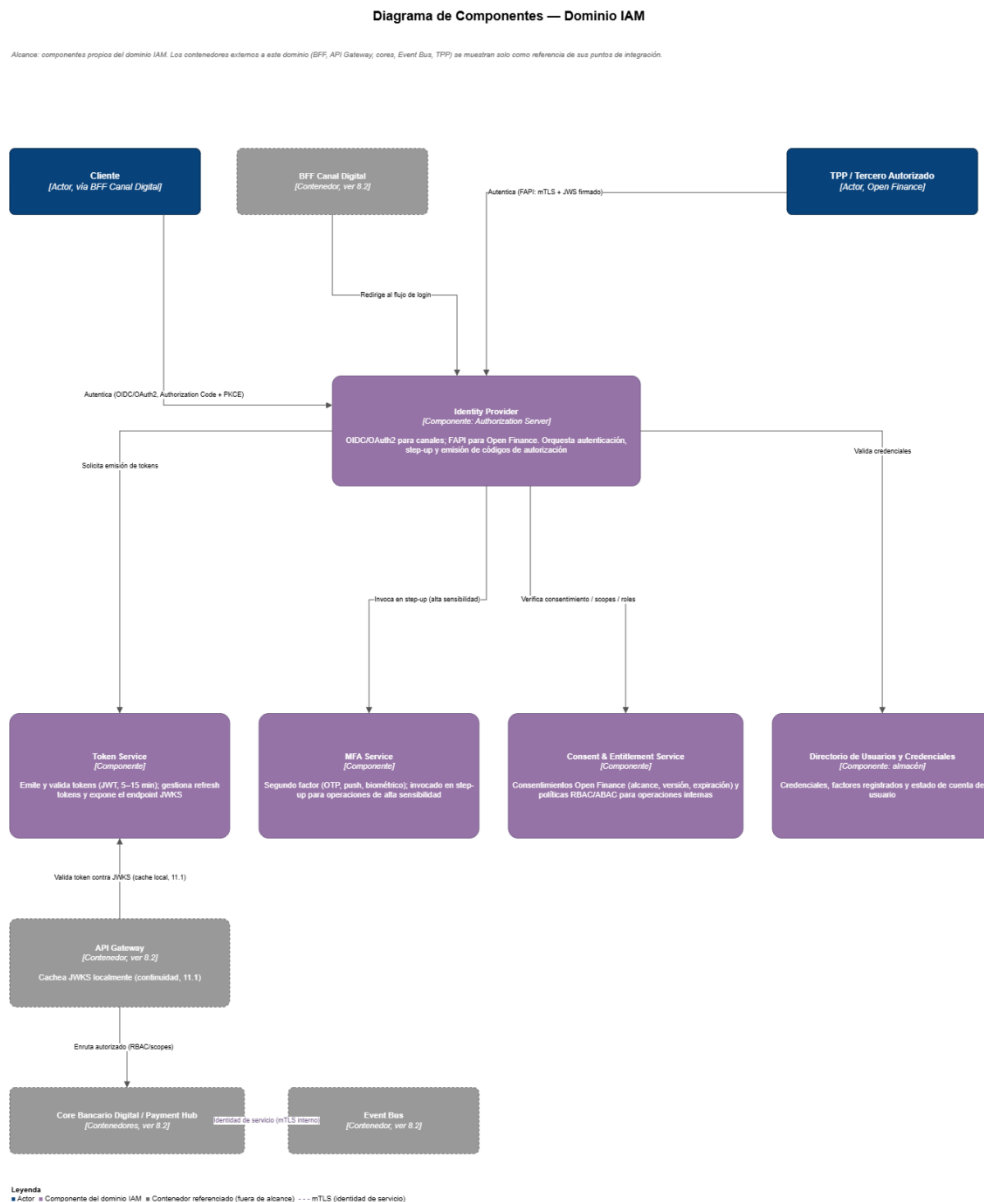


Figura 7. Diagrama de Componentes: Dominio IAM.

- **Identity Provider:** actúa como Authorization Server OIDC/OAuth2 para los canales propios y como servidor FAPI para Open Finance; orquesta el resto de los componentes en vez de implementar sus reglas directamente.
- **Token Service:** componente separado del Identity Provider; emite y valida los JWT de corta duración y expone el endpoint JWKS que el API Gateway cachea localmente (11.1); esta separación permite que el Gateway siga validando tokens ya emitidos aunque el Identity Provider esté temporalmente inaccesible.
- **MFA Service y Consent & Entitlement Service:** aíslan dos reglas de negocio distintas: el segundo factor (step-up) y el consentimiento/alcance de Open Finance, como componentes independientes, para que cambien de proveedor o de política sin tocar el Identity Provider.
- **Directorio de Usuarios y Credenciales:** almacén de credenciales y factores registrados; no se expone directamente a ningún contenedor fuera del dominio IAM: solo el Identity Provider lo consulta.

11.3 Diagrama de Secuencia: Autenticación y Autorización

Complementa el diagrama de componentes con el orden real de las interacciones para los dos flujos de la sección 11: el inicio de sesión de canal (OIDC/OAuth2 con step-up de MFA) y el flujo Open Finance para un TPP (tercero autorizado) mediante FAPI, con verificación de consentimiento. Ambos convergen en el mismo Identity Provider y en la misma validación de token en el API Gateway.

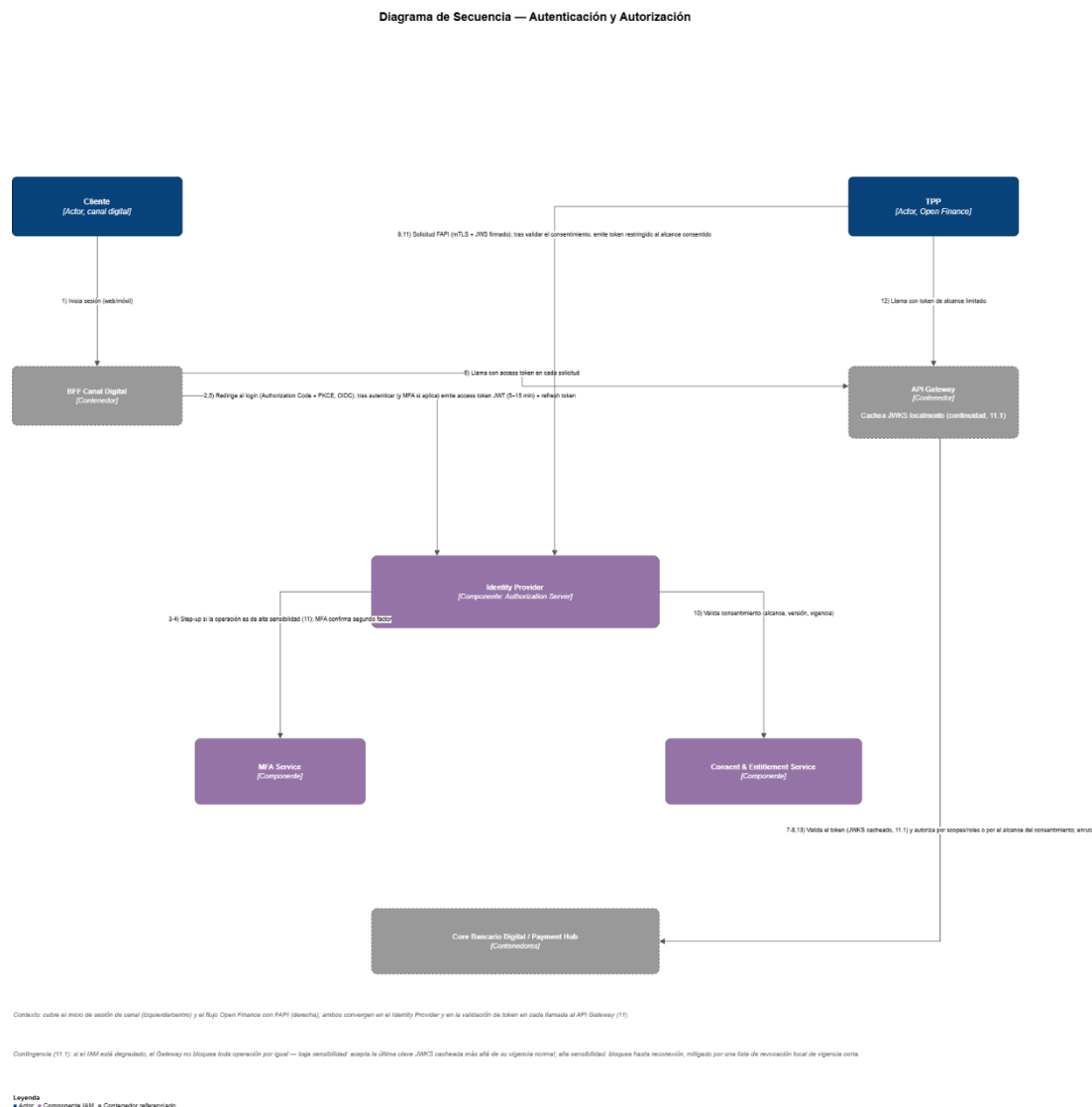


Figura 8. Diagrama de Secuencia: Autenticación y Autorización (canal digital y Open Finance).

- **Pasos 1–2:** el Cliente inicia sesión vía BFF Canal Digital, que redirige al Identity Provider (Authorization Code + PKCE, OIDC).
- **Pasos 3–4 (condicional):** si la operación solicitada es de alta sensibilidad (11), el Identity Provider invoca al MFA Service antes de continuar; de lo contrario se omite.
- **Paso 5:** el Identity Provider emite el access token (JWT, 5–15 min) y el refresh token.
- **Pasos 6–8:** el BFF llama al API Gateway con el access token en cada solicitud; el Gateway valida el token contra el JWKS cacheado localmente (11.1) y autoriza por scopes/roles (RBAC) antes de enrutar al Core Bancario Digital o al Payment Hub.
- **Pasos 9–11:** en paralelo, un TPP autenticado con FAPI (mTLS + JWS firmado) es dirigido al Identity Provider, que valida el consentimiento vigente con el Consent & Entitlement Service (alcance, versión, expiración) antes de emitir un token restringido a ese alcance; nunca un token de alcance completo.
- **Pasos 12–13:** el TPP llama al API Gateway con ese token de alcance limitado; el Gateway lo valida y autoriza de la misma forma que el flujo de canal, pero restringido a los scopes del consentimiento.
- **Contingencia (11.1):** el Gateway no aplica el mismo criterio de bloqueo a toda operación cuando el IAM está degradado: distingue por sensibilidad, como se detalla en el punto anterior; este diagrama muestra dónde ocurre esa validación (Gateway) y no en el Identity Provider, que puede estar inaccesible en ese momento.

12. Seguridad, Cumplimiento Normativo y Protección de Datos

El ejercicio se resuelve para Ecuador, bajo la Ley Orgánica de Protección de Datos Personales (LOPD). Esta sección identifica las obligaciones exigidas por la LOPDP y dónde se materializa cada una en la arquitectura. No sustituye una validación legal formal: se recomienda revisión por el equipo de cumplimiento/legal del banco antes de certificar conformidad.

12.1 Obligaciones LOPDP y su mecanismo en el diseño

Obligación LOPDP	Mecanismo en la arquitectura
Base de legitimación del tratamiento (consentimiento, ejecución contractual u obligación legal, según el caso)	Cada llamada al API Gateway lleva asociada la base de legitimación del dato tratado; el consentimiento explícito (incl. Open Finance) se captura y versiona en la Plataforma IAM (única fuente autoritativa de consentimiento en toda la plataforma, sección 7)
Registro de Actividades de Tratamiento (RAT)	Cada dominio (Originación, Pagos, Riesgo, Fraude, KYC) documenta qué datos personales trata, con qué finalidad y por cuánto tiempo, como parte de su contrato de dominio publicado en el catálogo (sección 14)
Evaluación de Impacto (EIPD) para tratamientos de alto riesgo	El scoring de riesgo crediticio y el scoring de fraude califican como tratamiento de alto riesgo (decisión con efecto significativo sobre la persona) y requieren EIPD antes de operar en producción
Decisiones automatizadas con efecto significativo	Riesgo y Fraude no pueden rechazar/bloquear de forma puramente automática sin vía de revisión: toda decisión adversa (rechazo de crédito, bloqueo de pago por fraude) genera un caso con posibilidad de revisión humana, no solo un score
Notificación de incidentes de seguridad	El Event Bus y el logging centralizado permiten reconstruir el alcance de un incidente (qué datos, qué sistemas, qué período) como insumo para la notificación a la autoridad de control y a los titulares en el plazo que exige la LOPDP; el plazo exacto debe confirmarse con el equipo legal vigente
Transferencias internacionales de datos	Motor de Riesgo y Motor de Fraude son vendors externos: si procesan datos fuera de Ecuador, requieren cláusulas contractuales de transferencia (o mecanismo equivalente reconocido) antes de integrarse; se valida en el proceso de alta de proveedor, no solo en el diseño técnico
Derechos del titular (acceso, rectificación, eliminación, oposición, portabilidad)	Expuestos como capacidad de "Gestión de Derechos del Titular" (no incluye consentimientos, que permanecen en IAM; ver nota de la primera fila), implementada como servicio dentro de la Plataforma KYC existente (capítulo 7), no un sistema nuevo. El API Gateway enruta la solicitud entrante hacia ese servicio (ver diagrama de contenedores, 8.2); la orquestación (recolectar el dato de cada sistema de registro propietario, incluida la Plataforma IAM para lo relativo a consentimiento, consolidar y responder al titular) la ejecuta el propio servicio de Cumplimiento, coherente con que el Gateway no coordina procesos multidominio (secciones 9 y 14)
Minimización y limitación de finalidad	El BFF y el API Gateway exponen contratos de API con "vistas" específicas por consumidor, nunca el modelo completo del core

Responsable y SLA (derechos del titular): el equipo de Cumplimiento es propietario del servicio de Gestión de Derechos del Titular y del SLA de respuesta al titular; cada solicitud genera un caso trazable (mismo mecanismo de correlación que el resto de la plataforma, sección 17) desde su ingreso hasta su cierre.

Responsable y procedimiento (incidentes de seguridad): el equipo de Seguridad/SRE es propietario de la detección (vía alertas de la Plataforma de Observabilidad, sección 17) y contención inicial; ante indicio de exposición de datos personales, escala de inmediato a Cumplimiento/DPO, quien determina si corresponde notificar a la Superintendencia de Protección de Datos Personales y a los titulares, y en qué plazo. La reconstrucción del alcance del incidente se apoya en el logging estructurado con correlation ID (sección 17).

12.2 Controles técnicos de soporte

- **Cifrado en tránsito:** TLS 1.2+ en todos los puntos de integración (Cliente–BFF, BFF–Gateway, Gateway–dominios, dominios–externos).
- **Cifrado en reposo:** AES-256 en cada sistema de registro; cifrado de payloads sensibles en el Event Bus.

- **Pseudonimización:** datos identificativos tokenizados en el Event Bus y en la Vista Agregada de Cliente; la re-identificación solo es posible en el sistema de registro autoritativo.
- **Auditoría:** todo acceso a datos personales queda registrado (quién, qué campo, cuándo, finalidad), no solo en el API Gateway, sino también en las llamadas dominio-a-dominio (p. ej., Payment Hub → Core, Core → Motor de Riesgo) y en los consumidores de eventos del Event Bus, que acceden a datos personales sin pasar por el Gateway. Cada dominio y cada consumidor de eventos instrumenta el mismo estándar de logging de acceso, correlacionado por el mismo mecanismo de correlation ID (sección 17).
- **Retención:** políticas de retención definidas por dominio (p. ej., transacciones de pago retenidas según normativa financiera, casos de fraude retenidos según política de riesgo), con purgado automático en cada sistema de registro, y también en las proyecciones y copias derivadas (Vista Agregada de Cliente, logs estructurados con datos personales, Historial de Interacción en la Plataforma de Observabilidad), que no quedan exentas solo por ser de solo lectura o por vivir en una plataforma primariamente técnica.

13. Estrategia de APIs y Mensajería

13.1 APIs

- **APIs síncronas (REST/HTTPS + JSON):** para operaciones que requieren respuesta inmediata (saldo, autenticación, screening de fraude).
- **APIs externas (Open Finance):** siguiendo estándar FAPI / especificaciones Open Finance, versionadas (/v1/, /v2/) con período de convivencia de 6–12 meses entre versiones antes de deprecarse.

Ejemplo de contrato: POST /v1/payments (Payment Hub)

```
{
  "idempotencyKey": "b3f1e2a0-3c1d-4e9a-8b21-...",
  "debtorAccountId": "ACC-DIG-00019284",
  "creditorAccountId": "ACC-LEG-00445190",
  "amount": { "value": 250.00, "currency": "USD" },
  "paymentType": "INTERNAL_TRANSFER",
  "initiatedBy": { "channel": "MOBILE_APP" }
}
```

La clave de idempotencia es obligatoria y la genera el cliente (app/BFF); el Payment Hub la rechaza si falta.

Otros endpoints (resumen)

- GET /v1/accounts/{id}/balance (Core Digital, vía Gateway): consulta saldo
- POST /v1/risk/credit-assessment (Motor de Riesgo): solicita evaluación de riesgo
- POST /v1/fraud/screen (Motor de Fraude): solicita score de fraude en tiempo real
- POST /openfinance/v1/consents (IAM): crea consentimiento AISP/PISP

13.2 Mensajería / Eventos y el patrón Outbox

El riesgo central de una arquitectura orientada a eventos en pagos es el problema de "dual write": si el servicio escribe su estado en la base de datos y luego publica el evento como dos operaciones separadas, una puede tener éxito y la otra fallar (evento perdido o evento huérfano sin estado real detrás).

Solución (Transactional Outbox): el estado del pago y el evento a publicar se escriben en la misma transacción de base de datos, en una tabla "outbox" dentro del propio Payment Store. Un proceso Outbox Publisher (o Debezium/CDC sobre esa tabla) lee la tabla de forma asíncrona y publica al Event Bus con reintentos hasta confirmar, marcando la fila como publicada. Si el proceso falla a mitad de camino, el evento sigue pendiente en la tabla y se reintenta; nunca se pierde ni se duplica de forma silenciosa (ver diagrama de componentes, sección 8.3).

Deduplicación del lado del consumidor (condiciones precisas): la fórmula "at-least-once + deduplicación = exactly-once" solo es válida si se cumplen tres condiciones; enunciarla sin ellas es insuficiente:

- **1. Registro de deduplicación + efecto local en una misma transacción, cuando sea posible:** si el efecto del evento es una escritura local (p. ej., actualizar la Vista Agregada), el consumidor inserta el eventId en su propia tabla de "eventos procesados" y aplica el efecto en la misma transacción de base de datos. Si cualquiera de las dos falla, ambas se revierten; nunca queda un efecto aplicado sin su registro de dedup, ni viceversa.

- **2. Para llamadas remotas (el efecto no es una escritura local sino invocar otro sistema, p. ej. Payment Hub llamando a debitar en un core):** no existe una transacción compartida entre Payment Hub y el core, por lo que el registro local de "ya procesado" no es suficiente por sí solo. Se requiere una clave de operación estable (operationId) enviada en cada llamada, y el sistema remoto debe soportar consulta de estado por esa clave (ver el procedimiento de resultado incierto más abajo).
- **3. Solicitudes concurrentes con la misma clave:** si dos solicitudes llegan con el mismo idempotencyKey mientras la primera sigue en curso, la segunda no se procesa en paralelo: el Idempotency Guard la rechaza (o la hace esperar) según el estado registrado (PENDING/COMPLETED/FAILED) para esa clave, evitando una carrera que dos ejecuciones concurrentes sí producirían.

Con estas tres condiciones, Transactional Outbox sigue siendo la base correcta, pero vale notar su límite con precisión: outbox ofrece entrega eventual con reintentos, condicionada a la conservación del dato en la tabla outbox y a la recuperación del publicador o del broker; no es una garantía absoluta de "nunca cero veces" (una pérdida irrecoverable de la fila outbox, o un publicador que nunca se recupera, sí puede perder el evento). Permite además duplicados en el broker (reintentos de red, rebalanceo del consumidor) y por eso exige consumidores idempotentes según las condiciones anteriores, no los sustituye.

- **Esquema versionado:** registro de esquemas (schema registry) con JSON Schema o Avro.
- **Reintentos y DLQ:** backoff exponencial en el consumidor; tras N intentos fallidos el evento va a una Dead Letter Queue para revisión manual, sin bloquear el resto del tópico.
- **Ordenamiento:** partición por ID de cuenta/cliente cuando el orden de eventos de una misma entidad importa.
- **Política de pausa ante DLQ (pausa de partición completa):** enviar un evento a DLQ y continuar con el resto del tópico no es suficiente por sí solo: permitiría que un evento posterior de la misma cuenta adelante al que falló, rompiendo el orden de esa entidad. Kafka no ofrece de forma nativa "saltar una clave y seguir con las demás" dentro de una partición sin un mecanismo adicional de almacenamiento diferido; en vez de construir esa capa extra, se opta por pausar la partición completa donde ocurrió el fallo hasta resolver el evento en DLQ. Esto es una decisión consciente, no una limitación oculta: como el particionado es por cuenta (sección 13.2), pausar una partición bloquea también las demás cuentas asignadas a esa misma partición, no solo la cuenta afectada; el tamaño de ese radio de impacto depende del número de particiones configuradas, y es un parámetro a dimensionar según el volumen esperado, no algo que este documento fije de antemano.

13.3 Resultado incierto de una llamada remota (timeout en un débito)

Caso concreto: Payment Hub llama a Ledger Adapter para debitar una cuenta; el core la procesa correctamente, pero la respuesta se pierde por una falla de red antes de llegar a Payment Hub. Si Payment Hub reintenta con una llamada de débito ordinaria, el riesgo es debitar dos veces. El outbox de Payment Hub no cubre este caso: outbox resuelve la publicación de eventos propios, no la incertidumbre de una llamada síncrona saliente hacia otro sistema.

- Toda operación de movimiento de fondos lleva un operationId estable, generado por Payment Hub y conservado en todos sus reintentos. Cada paso de una orden (reserva, débito, crédito o compensación) tiene un identificador de operación distinto, vinculado al mismo paymentId. Antes de ejecutar efectos externos, Payment Hub persiste la relación entre el cliente o aplicación solicitante, el idempotencyKey, el paymentId, los datos de la orden y los operationId correspondientes. La misma clave de idempotencia dentro del mismo ámbito identifica una única orden; si se reutiliza con datos diferentes, la solicitud se rechaza
- **Ante timeout, Payment Hub NO reintenta a ciegas:** primero consulta el estado de esa operación en el core (GET por operationId).
- **Si el core confirma que se aplicó:** Payment Hub la trata como exitosa y continúa el esquema, sin reintentar el débito.
- **Si el core confirma que no se aplicó (o no la encuentra):** es seguro reintentar con el mismo operationId, porque el core debe tratarlo como idempotente.
- **Si el estado sigue sin poder determinarse tras reintentar la consulta:** el esquema no avanza: el caso pasa a verificación manual (cola de reconciliación), y los fondos permanecen en estado de retención hasta resolverse; nunca se asume éxito ni se libera la retención por defecto.

Requisito arquitectónico derivado: tanto el Core Digital como el Core Tradicional deben exponer, para las operaciones de movimiento de fondos, (a) un parámetro de operationId idempotente y (b) un endpoint de consulta de estado por esa clave. El Core Digital puede diseñarse así desde el inicio; el Core Tradicional, al ser legado, puede no soportarlo de forma nativa; en ese caso se requiere una capa anticorrupción (patrón Anti-Corruption Layer, sección 9) que añada ambas

capacidades delante de su API antes de integrarlo al esquema de pagos. Sin esa capa, el Core Tradicional no debería recibir llamadas de débito/crédito desde Payment Hub sin control adicional de reconciliación manual.

Esa capa se modela como un componente propio dentro de Payment Hub, "ACL Core Tradicional" (ver diagrama de componentes, 8.3), y no como parte de Ledger Adapter: Ledger Adapter mantiene una lógica genérica de débito/crédito válida para cualquier core, mientras el ACL concentra exclusivamente las particularidades del legado (traducción de protocolo, adición de operationId, consulta de estado). Ledger Adapter llama al Core Digital directamente y al Core Tradicional únicamente a través del ACL.

Precondición que el ACL no puede suplir por sí solo: agregar un adaptador no garantiza automáticamente idempotencia: el ACL solo puede ofrecer operationId y consulta de estado si el Core Tradicional expone, de alguna forma, una manera confiable de identificar la operación ya ejecutada (p. ej., una referencia externa que el legado persista y permita consultar, o un mecanismo equivalente). Si el legado no ofrece nada de eso, el ACL no puede inventarlo: en ese caso el resultado de un timeout se mantiene incierto por diseño y va siempre a conciliación manual, en vez de presentarse como un caso resuelto por la sola existencia del adaptador. Confirmar esta capacidad del Core Tradicional específico es una validación pendiente antes de comprometerse con este diseño.

Ejemplo de evento: payment.confirmed.v1

```
{
  "eventId": "9a6e7c3b-...",
  "eventType": "payment.confirmed.v1",
  "correlationId": "req-7f2a...",
  "occurredAt": "2026-09-06T14:32:10Z",
  "paymentId": "PAY-00058231",
  "debtorAccountId": "ACC-DIG-00019284",
  "creditorAccountId": "ACC-LEG-00445190",
  "amount": { "value": 250.00, "currency": "USD" },
  "status": "CONFIRMED"
}
```

Otros eventos (resumen)

- account.opened.v1
- payment.initiated.v1 / payment.failed.v1
- fraud.case.created.v1
- legacy.party_reference.updated.v1 (CDC desde el core tradicional, referencia/vínculo cuenta-parte, no perfil autoritativo: IAM sigue siendo la única fuente de perfil, sección 7)

14. Modelo de Gobierno de APIs y Microservicios

- **Catálogo centralizado:** el API Gateway/API Manager es la fuente única de verdad, con portal de desarrolladores para las APIs Open Finance.
- **Ciclo de vida API-first:** diseño (contrato OpenAPI/AsyncAPI revisado antes de construir) → build → test → publish → deprecate, con revisión de un comité de arquitectura/seguridad/producto.
- **Estándares obligatorios:** nomenclatura /v{n}/{recurso}, manejo uniforme de errores (RFC 9457, que reemplaza a RFC 7807), paginación estándar, versionado semántico.
- **Ownership:** cada dominio/microservicio tiene un equipo propietario responsable de su API y sus SLAs ("you build it, you run it").
- **Reglas de microservicios:** ningún servicio accede a la base de datos de otro; los cambios de contrato son backward-compatible o pasan por período de convivencia; todo contrato se publica en el catálogo antes de integrarse.
- **Límite Gateway vs. orquestación de negocio:** el API Gateway enruta y aplica políticas técnicas (authN/Z, rate-limiting, versionado de contrato); coordinar los pasos de un proceso de negocio (p. ej., el esquema de pago) es responsabilidad del dominio propietario del proceso (Payment Hub), nunca del Gateway. Esta separación se revisa en cada diseño de API que pase por el comité de gobierno.
- **Métricas de gobierno:** adherencia a SLA, % de APIs con contrato publicado, tiempo de vida promedio de una versión deprecada.

14.1 Ciclo de vida y versionado

Precisa el ciclo de vida ya enunciado (14) con reglas de versionado y retiro verificables, no solo declaradas.

- **Versionado semántico:** MAJOR.MINOR.PATCH a nivel de contrato. Un cambio MINOR o PATCH (campo opcional agregado, corrección de descripción, relajar una validación) no rompe consumidores y no cambia la URI. Un cambio MAJOR (campo requerido nuevo, eliminación o renombre de campo, cambio de tipo) exige una nueva URI `/v{n+1}/{recurso}` y coexistencia con la versión anterior.
- **Coexistencia y retiro:** una versión MAJOR deprecada permanece activa un mínimo de 6 meses o hasta que el consumo residual caiga por debajo de un umbral acordado con los consumidores, lo que ocurra después. Se anuncia con encabezado Deprecation (RFC 9745) y Sunset (RFC 8594) en cada respuesta, y se monitorea el tráfico por versión (17) para no retirar una versión aún en uso significativo.
- **Gate de diseño:** ningún endpoint se implementa antes de que su contrato (OpenAPI 3.1 para REST, AsyncAPI 2.6 para eventos, SDL para el GraphQL del BFF) sea aprobado por el comité de gobierno (14.4) y publicado en el catálogo (14.2). El build que no corresponde a un contrato aprobado no pasa el pipeline de CI (14.2).

14.2 Catálogo, contratos y registro de esquemas

El catálogo (14) no es solo un directorio: es la fuente que valida compatibilidad de forma automática, antes de que un cambio llegue a producción.

- **Qué se registra:** el contrato OpenAPI de cada API REST (incluye el ejemplo de 13.1), el esquema AsyncAPI de cada tópico de eventos del Event Bus (13.2, 13.3) y el SDL del GraphQL expuesto por el BFF; cada uno con dominio propietario, nivel de criticidad (14.5) y estado (draft, publicado, deprecado, retirado).
- **Registro de esquemas para eventos:** cada tópico de Kafka tiene un esquema versionado en modo de compatibilidad BACKWARD: un productor no puede publicar un esquema que rompa a los consumidores existentes del mismo tópico. El Outbox Publisher (13.2) valida contra el registro antes de publicar; una violación de compatibilidad bloquea el despliegue, no la publicación en caliente.
- **Pruebas de contrato en CI/CD:** cada pull request que modifica un contrato ejecuta una comparación de esquema (diff estructural OpenAPI/AsyncAPI) contra la versión publicada; un cambio incompatible sin bump de versión MAJOR falla el pipeline. Los consumidores internos críticos (p. ej. BFF sobre Payment Hub) mantienen pruebas de contrato dirigidas por el consumidor (consumer-driven contract testing) que se ejecutan contra cada cambio del productor antes de publicar.

14.3 Seguridad y cuotas en el Gateway

Complementa 11 (IAM) y 19.1 (controles de seguridad) con las políticas que el Gateway aplica de forma uniforme a todo el tráfico, independientemente del dominio.

- **Autenticación y autorización por scope:** cada endpoint declara los scopes OAuth2 requeridos en su contrato; el Gateway valida el token (JWT, OAuth2/OIDC, sección 11) y rechaza con 403 antes de enrutar si el scope no está presente, sin delegar esa validación al dominio.
- **mTLS interno:** toda llamada entre el Gateway y los contenedores internos, y entre contenedores internos entre sí, usa mTLS (9.1); el Gateway no reenvía tráfico en texto plano ni confía en la red interna por defecto.
- **Cuotas y limitación de tasa:** límites por consumidor y por API, diferenciados por tipo de consumidor: canal propio (Web/Móvil vía BFF) recibe cuota alta orientada a disponibilidad; terceros Open Finance (TPP) reciben cuota contractual explícita, sostenida y de ráfaga, acorde a FAPI. Al exceder la cuota, el Gateway responde 429 con encabezado Retry-After; no degrada el servicio de otros consumidores (aislamiento por cuota).
- **WAF y protección perimetral:** reglas de WAF (ver también 8.5, Azure Front Door) para los patrones de ataque comunes a APIs (inyección, payloads anómalos, abuso de paginación) se aplican antes del Gateway, no lo sustituyen.

14.4 Gobernanza organizacional

El comité mencionado en 14 (revisión de ciclo de vida) se precisa aquí como una instancia con composición, cadencia y autoridad de decisión explícitas. Sin esto, "revisión de un comité" no es verificable.

- **Composición:** un arquitecto de APIs (preside), un representante de seguridad, el equipo propietario del dominio que propone el cambio, y un representante de un dominio consumidor relevante cuando el cambio afecta contratos existentes.
- **Qué requiere revisión del comité:** todo contrato nuevo antes de su primera publicación; todo cambio MAJOR a un contrato existente (14.1); toda excepción a los estándares obligatorios (14, nomenclatura, RFC 9457, versionado). Los cambios MINOR/PATCH no requieren comité, solo pasan el pipeline de compatibilidad (14.2).
- **Mecanismo:** propuestas como RFC corto (contexto, contrato propuesto, impacto en consumidores existentes, plan de migración si aplica), con decisión registrada (aprobado, aprobado con cambios, o rechazado) y trazable junto al contrato en el catálogo (14.2).

- **Cadencia:** revisión semanal para cambios ordinarios; vía expedita (48 horas) para correcciones de seguridad o incidentes en producción, documentada retroactivamente en la siguiente sesión ordinaria.

14.5 SLA/SLO por nivel de criticidad

Las métricas de gobierno de 14 (adherencia a SLA) requieren un SLA definido por nivel, no un único objetivo global. La siguiente tabla fija los objetivos de referencia; se validan y ajustan con datos reales de producción y con los simulacros de 15.

Nivel	Ejemplos	Disponibilidad objetivo	Latencia (p99)	Ventana de mantenimiento
Crítico	Payment Hub (13.3), Core Digital, IAM (autenticación)	99.9%	< 2 s (aceptación del pago, 13.3)	Sin ventana; despliegues con blue/green o canary
Estándar	Vista Agregada de Cliente, consultas de producto, KYC	99.5%	< 1 s	Ventana semanal acordada, fuera de horario pico
Interno	APIs de soporte, reportería, batch no expuesto a canal	99.0%	Sin objetivo estricto (asíncrono)	Ventana flexible, coordinada con el equipo propietario

El nivel Crítico usa presupuesto de error (error budget): superar el 0.1% de indisponibilidad mensual permitido detiene nuevos despliegues de ese dominio hasta estabilizar, salvo corrección del propio incidente; la misma disciplina que 15 aplica a los objetivos de RTO/RPO.

15. Alta Disponibilidad y Recuperación ante Desastres

15.1 Componentes y Estrategia HA/DR

La estrategia se define por nivel de criticidad de cada contenedor y dependencia externa, no de forma genérica:

Componente	Criticidad	Estrategia HA/DR
BFF / API Gateway	Alta (punto único de entrada)	Stateless: la sesión vive en el JWT emitido por IAM, no en memoria del BFF; multi-AZ, autoscaling, sin estado de servidor que recuperar
Core Bancario Digital	Alta	Despliegue multi-AZ con réplica síncrona para proteger los movimientos confirmados ante fallos de zona. Recuperación en una región secundaria mediante réplica asíncrona. RTO objetivo: 15 minutos. Ante pérdida de región, el RPO depende del retraso de replicación y debe medirse y validarse mediante simulacros. Antes de reanudar operaciones, se concilian los movimientos del período afectado. Esta estrategia protege la persistencia contable del Core Digital y es independiente de la recuperación de órdenes de Payment Hub.
Payment Hub	Alta	Despliegue multi-AZ y recuperación en una región secundaria. RTO objetivo: 15 minutos. RPO objetivo: cero para el registro de órdenes y operaciones, mediante replicación síncrona aislada del Payment Store (Data Guard, modo Maximum Availability) hacia la región de recuperación, con fail-closed sobre nuevas órdenes si esa confirmación se degrada (15.1); hasta un minuto para el estado de ejecución reconstruible, en replicación asíncrona. Tras un desastre, conciliar con los cores y rieles antes de reanudar los pagos afectados. Objetivos sujetos a validación mediante simulacros (15.3).

Componente	Críticidad	Estrategia HA/DR
Registro de Propiedad de Cuentas	Alta (bloquea el enrutamiento de todo el tráfico si no está disponible)	Réplica multi-AZ del almacén clave-valor. Fallar cerrado (sección 7.1) define el comportamiento MIENTRAS dura el fallo; no sustituye un objetivo de recuperación: RTO objetivo ~5 min, RPO objetivo ~0 (referencial, pendiente de validar por simulacro; propietario: equipo de plataforma de migración). Procedimiento de recuperación: al restaurar el registro (backup o réplica), se audita contra el estado real de cada core (propietario, versión, habilitación local) y contra el registro durable de migración (sección 7.1/16.1): un indicador de habilitación local, por sí solo, no determina la propiedad; la reconciliación cruza esas tres fuentes antes de reabrir enrutamiento, en vez de asumir que su propio backup está actualizado
Core Bancario Tradicional (externo)	Alta: sigue siendo autoritativo de productos no migrados	Su HA/DR la gestiona el equipo propietario del legado; se exige un SLA de disponibilidad y un RTO/RPO documentado como precondition de integración, no un objetivo que la nueva plataforma pueda garantizar por sí sola
Plataforma IAM (externo)	Alta: sin autenticación no hay acceso a ningún flujo	Se exige alta disponibilidad multi-región al proveedor; el API Gateway cachea localmente las claves públicas de verificación de JWT (JWKS) con TTL corto, para tolerar una degradación breve del IAM sin bloquear la validación de tokens ya emitidos
Event Bus	Alta (backbone)	Clúster replicado (factor ≥ 3) con <code>acks=all</code> y <code>min.insync.replicas≥ 2</code> , particionado por dominio, multi-AZ. Sin mirroring activo a la región DR (8.5); la región secundaria levanta un clúster vacío y la recuperación es por categoría de evento, no por reproducción de historial.
Vista Agregada de Cliente	Media	RPO más laxo: se resincroniza por backfill contra cada sistema de registro propietario si se pierde, no reproduciendo el historial de eventos (8.5).
Motor de Riesgo, Motor de Fraude, KYC (externos)	Depende de SLA del proveedor	Circuit breaker + degradación controlada (nunca fallo abierto sin control en pagos)

Sobre los RTO/RPO: los valores indicados son objetivos de diseño, no garantías demostradas; se validan mediante simulacros de DR periódicos (15.3) y se ajustan según sus resultados.

Conciliación tras un failover de Payment Hub: para recuperar una orden no basta con comprobar que hubo un movimiento contable. Se necesita conservar su identidad, los datos originales y la relación con cada operación ejecutada. Payment Hub mantiene esta información en un registro durable de órdenes y operaciones dentro de su almacenamiento propio. La recuperación de este registro es una precondition para reanudar los pagos afectados.

La identidad y los datos de una orden aceptada deben quedar protegidos frente a la pérdida de la región principal antes de ejecutar movimientos externos. Para ese registro se establece un objetivo de RPO cero: el Payment Store se aísla del resto del estado de Payment Hub y se replica hacia la región secundaria con transporte síncrono (Oracle Data Guard, modo Maximum Availability), mientras que Core Digital y el resto del estado técnico permanecen en replicación asíncrona por ser reconstruibles desde eventos o backfill (8.5). Si la confirmación síncrona del destino no llega dentro del tiempo configurado, por standby degradado o inalcanzable, Payment Hub deja de aceptar nuevas órdenes mientras dure la degradación: falla cerrado sobre el registro de órdenes en vez de aceptarlas sin la garantía de durabilidad cross-región, siguiendo el mismo principio de fail-closed aplicado al IAM (11.1). El RPO referencial de un minuto aplica al estado de ejecución reconstruible, no a la información necesaria para identificar las órdenes aceptadas. Esta distinción debe reflejarse en la configuración de persistencia y replicación de Payment Hub, y su cumplimiento se valida mediante simulacros de DR (15.3).

Cada core conserva el `operationId` como referencia del movimiento y permite consultar su resultado, sujeto a la precondition del legado descrita en 13.3. Tras un failover, el proceso de conciliación recupera las órdenes y sus operaciones, consulta los resultados en los cores y reconstruye el estado de ejecución. Para pagos externos, también

verifica el resultado con el riel correspondiente mediante su referencia de operación. Un resultado incierto permanece pendiente de conciliación; no se interpreta como una operación no ejecutada ni se reintenta a ciegas.

Durante este proceso se bloquea el reprocesamiento de las órdenes afectadas y se mantienen las restricciones necesarias sobre sus cuentas hasta resolver las operaciones pendientes. Una transformación determinística puede utilizarse para generar identificadores reproducibles, pero no se presupone reversible ni sustituye el registro durable de las relaciones entre identificadores.

15.2 Escenarios de Contingencia

El punto anterior define, por componente, una estrategia y un objetivo de RTO/RPO. Esta sección detalla cinco escenarios de falla concretos: qué se detecta, qué respuesta es automática y cuál requiere intervención, y el procedimiento de recuperación paso a paso. No son alternativas entre sí: un mismo incidente real puede combinar más de uno (por ejemplo, una pérdida de región que además deja el Registro de Propiedad de Cuentas en un estado a auditar).

Escenario A: Pérdida de una zona de disponibilidad (AZ) en la región primaria

- **Detección:** health checks fallidos en la AZ afectada (probes de Azure Front Door y de AKS).
- **Respuesta automática:** el tráfico se redirige a las AZ sanas sin intervención manual para las rutas de enrutamiento (Front Door/API Management). AKS es zone-redundant, con margen dimensionado en cada zona (redundancia N+1) para absorber la pérdida de una AZ sin degradar el servicio; si el margen no basta, el autoscaler agrega capacidad en las zonas sanas en minutos. Oracle Database@Azure replica de forma síncrona entre AZ2 y AZ3 mediante Data Guard en modo SYNC, con Fast-Start Failover (observer) para la conmutación de rol; no se usa RAC aquí, que es clustering de almacenamiento compartido, no replicación. Azure Cache for Redis también es zone-redundant y síncrona entre zonas, pero la reconexión del cliente tras un failover no planificado no es instantánea (8.5); las aplicaciones deben reintentar con backoff en vez de asumir continuidad de conexión. No hay estado de sesión en el BFF/API Gateway que deba recuperarse, porque la sesión vive en el token emitido por IAM (15.1).
- **RTO/RPO:** por componente, no un valor único: Oracle Database@Azure, RPO 0 (transporte síncrono), RTO del orden de segundos (detección del observer más conmutación de rol); Redis, RPO≈0 (réplica síncrona entre zonas), RTO de segundos a bajas decenas de segundos según la reconexión del cliente; Kafka, RPO 0 solo si los productores usan acks=all y min.insync.replicas≥2, RTO del orden de segundos (elección de nuevo líder de partición). Esto es distinto de la réplica entre regiones (Escenario B), que es asíncrona salvo para el Payment Store de Payment Hub (15.1), que sí replica de forma síncrona entre regiones.
- **Procedimiento de recuperación:** al restituir la AZ caída, verificar la integridad de sus réplicas antes de reincorporarla al tráfico activo, no asumir que el estado replicado quedó consistente sin comprobarlo; conciliar cualquier solicitud que haya quedado en curso durante la transición, con el mismo principio de conciliación que aplica Payment Hub tras un failover (15.1); y restaurar la capacidad normal de autoscaling una vez confirmada la salud de la zona. El sistema ya opera de forma normal con las AZ restantes durante la falla.

Escenario B: Pérdida de la región primaria

- **Detección:** probes de Azure Front Door fallidos en toda la región primaria; alertas de Azure Monitor sostenidas en el tiempo (no un pico aislado, para evitar declarar un failover por una degradación breve).
- **Respuesta:** failover L7 en Azure Front Door hacia la región secundaria (Brazil Southeast, standby cálido, 8.5); este redireccionamiento de tráfico puede ser automático, basado en los probes sostenidos de la detección anterior. La habilitación de escritura en la región secundaria es un paso separado y no automático (ver el procedimiento de recuperación, más abajo): mientras no se autorice, la región secundaria puede recibir tráfico pero rechaza escrituras.
- **Impacto en los datos:** el resto de Oracle Database@Azure (Core Bancario Digital) y Azure Cache for Redis solo cuentan con réplica asíncrona hacia la región secundaria; el RPO real depende del retraso de esa replicación en el momento del incidente y debe medirse con simulacros (15.3), no asumirse cercano a cero. El Payment Store de Payment Hub replica de forma síncrona (15.1): si la confirmación estaba vigente en el momento de la pérdida, el registro de órdenes no se pierde; si la pérdida ocurre durante una degradación del enlace síncrono, aplica el fail-closed descrito en 15.1 en vez de asumir RPO cero por defecto. El Event Bus (Kafka) no tiene réplica multi-región activa: la región secundaria levanta un clúster vacío y la recuperación de eventos es por categoría, no por reproducción del historial (8.5); los eventos de interacción del canal sin consumir antes de la pérdida se pierden, sin fuente alterna.
- **Procedimiento de recuperación:** (1) confirmar pérdida sostenida de la región, no solo una degradación, con el criterio de detección ya descrito.
- (2) Seguridad/SRE autoriza de forma manual y explícita la promoción, incrementando la Generación de Región Activa: un contador versionado, almacenado en un punto de control independiente de la salud de cualquiera de las dos regiones (capa de control de Front Door), que Payment Hub, Core Digital y el Ledger Adapter validan antes

de procesar cualquier escritura. Este paso aísla de inmediato las escrituras del sitio anterior, aunque siga técnicamente operativo, sin depender de que sus propios probes hayan fallado.

- (3) promover a primarias las réplicas de Oracle Database@Azure (síncrona para el Payment Store, asíncrona para el resto) y de Redis en la región secundaria.
- (4) sobre el Registro de Propiedad de Cuentas recién promovido, ejecutar la misma auditoría cruzada de tres fuentes del Escenario D antes de confiar en su contenido: una promoción sin errores técnicos no garantiza que el registro refleje la propiedad real.
- (5) escalar AKS standby, que opera en capacidad reducida, a capacidad operativa.
- (6) levantar un clúster Kafka vacío en la región secundaria y reconstruir por categoría, no por replicación de historial: republicar eventos de pago desde el Payment Store; reanudar CDC del legado desde su propio log de cambios; aceptar la pérdida de eventos de interacción del canal sin fuente alterna; resincronizar vistas derivadas por backfill contra cada sistema de registro propietario.
- (7) ejecutar la conciliación de Payment Hub descrita en la introducción de este capítulo, y solo entonces reabrir tráfico de escritura de forma gradual (canary) antes de conmutar el 100% a la región secundaria; hasta ese punto, la región secundaria puede recibir tráfico de solo lectura o degradado, pero no escrituras.
- **Post-recuperación:** al restaurarse la región primaria, no se vuelve a ella automáticamente. El failback es una operación planificada, en ventana de mantenimiento, para evitar un segundo corte no controlado.

Escenario C: Falla o degradación del Core Bancario Tradicional (legado)

- **Detección:** timeouts o errores sostenidos del ACL Core Tradicional al invocar el legado (13.2, 13.3).
- **Respuesta:** las operaciones sobre cuentas aún no migradas reciben el mismo tratamiento de resultado incierto del ACL (13.3 y el escenario 2 de 8.4): no se asume éxito ni fallo, se marca la operación como pendiente de verificación. No existe una conmutación automática de ese tráfico hacia el Core Digital, porque las cuentas de uno y otro core no son intercambiables (pertenecen a productos distintos, según la regla de propiedad de 7.1).
- **Límite de responsabilidad:** el HA/DR del legado en sí lo gestiona su equipo propietario (15.1); esta plataforma solo puede aplicar backpressure o un circuit breaker para no saturar un legado ya degradado, encolar o rechazar operaciones según la criticidad definida para cada una, y escalar al equipo propietario contra el SLA de disponibilidad exigido como precondition de integración.
- **Procedimiento de recuperación:** al restablecerse el legado, la cola de operaciones pendientes se procesa según su estado, no de forma uniforme. Las operaciones rechazadas antes de llegar al legado se reintentan automáticamente con su mismo operationId, sin riesgo de duplicidad porque el legado nunca las procesó. Las operaciones con resultado incierto se consultan por operationId solo si el legado expone esa capacidad (13.3); si la expone, se reconcilian según lo que devuelva, si no, permanecen en conciliación manual y no se reintentan a ciegas. Las operaciones ya confirmadas por el legado antes de drenar la cola solo actualizan el registro de Payment Hub, sin reenviarse. La cola de verificación y las reservas de fondos asociadas tienen un tiempo máximo de espera definido (referencial, algunas horas según el impacto regulatorio); superado ese tiempo sin resolución, escala a conciliación manual con Cumplimiento/Operaciones en vez de permanecer indefinidamente en espera automática. Solo entonces se reabre el tráfico normal hacia esas cuentas.

Escenario D: Falla del Registro de Propiedad de Cuentas

- **Detección:** el API Gateway o el Ledger Adapter no completan la consulta de propietario, versión y estado (timeout o error).
- **Respuesta:** fail-closed (7.1): se rechazan las operaciones de escritura que dependan de esa consulta. No se enruta con un valor cacheado ni asumido, porque un corte de propiedad en curso (16.1) volvería incorrecta cualquier suposición sobre qué core es el propietario actual.
- **Procedimiento de recuperación:** al restaurar el registro, por backup o por promoción de una réplica, no se reabre el enrutamiento de inmediato. Primero se audita cruzando tres fuentes: el propio registro restaurado, el estado real de habilitación local en cada core, y el registro durable de migración (7.1/16.1), que documenta cortes en curso o interrumpidos. Si hay una discrepancia (por ejemplo, un corte que había llegado al paso 4 de 16.1 pero no al paso 5), se aplica el procedimiento de recuperación de un corte interrumpido (16.1) antes de reabrir esa cuenta en particular. El enrutamiento se reabre por segmento, no en bloque.

Escenario E: Corrupción de datos

- **Naturaleza distinta de los escenarios A y B:** no es una falla de disponibilidad sino de integridad. Una réplica síncrona o asíncrona propaga fielmente también el dato corrupto, de modo que el failover multi-AZ o multi-región de los escenarios anteriores no protege contra este caso.

- **Detección:** validaciones de integridad (checksums, conciliación periódica entre saldo y movimientos, alertas de anomalías). A diferencia de una pérdida de infraestructura, la detección no siempre es inmediata: puede ocurrir horas o incluso días después del hecho que originó la corrupción.
- **Contención:** aislar el componente afectado para no seguir propagando la corrupción; si el origen es una versión desplegada con un defecto, congelar nuevos despliegues hasta identificar la causa.
- **Recuperación:** restaurar desde un backup de un punto en el tiempo anterior a la corrupción, almacenado en una cuenta o ubicación independiente de la fuente y protegido con inmutabilidad (WORM o soft-delete con retención bloqueada), para que ni la corrupción ni un actor malicioso puedan alterarlo o eliminarlo. La retención mínima es de 35 días, calibrada para exceder el tiempo de detección observado en los simulacros de este escenario (15.3), no una afirmación genérica. El alcance de la restauración se acota a los datos o cuentas identificados en el diagnóstico, no es un rollback global salvo que el diagnóstico confirme alcance amplio. La reconstrucción posterior mediante replay del Event Bus o del registro durable correspondiente excluye explícitamente el cambio o evento que originó la corrupción, identificado en la fase de diagnóstico, con la misma lógica de conciliación descrita para el escenario B.
- **Diferencia clave frente al escenario B:** este caso exige retención de backups punto en el tiempo, no solo réplicas activas, y su RTO es naturalmente más largo porque incluye el diagnóstico de causa y alcance antes de poder restaurar.

15.3 Simulacros de Recuperación ante Desastres (Game Days)

Varios RTO/RPO de este capítulo están marcados como objetivo de diseño, no garantía demostrada. Esta sección define cómo se valida esa distinción en la práctica, en vez de dejarla como una salvedad implícita.

Escenario	Cadencia mínima	RTO / RPO objetivo	Responsable y criterio de aprobación
A. Pérdida de AZ	Trimestral	RTO < 30 s (conmutación intra-región). RPO 0 (Oracle y Redis síncronos entre zonas; Kafka con acks=all, min.insync.replicas≥2).	Plataforma/SRE. Sin pérdida de escritura confirmada (datos de prueba marcados); métricas de error vuelven a su valor base dentro del RTO objetivo; sin intervención manual.
B. Pérdida de región	Trimestral	RTO 15 min hasta servicio útil, incluyendo conciliación y reapertura (infraestructura y pagos medidos por separado). RPO 0 para el registro de órdenes de Payment Hub (Payment Store síncrono, 15.1); minutos para el resto de Oracle Database@Azure y Redis, medido y no asumido.	Seguridad/SRE autoriza la promoción; Plataforma/SRE la ejecuta. Pérdida de región simulada después de un débito confirmado sin esa orden perdida; partición de red con el primario aún operando sin escritura simultánea en ambos sitios; retorno del primario sin doble escritura ni propietarios simultáneos del Registro de Propiedad.
C. Falla del legado	Semestral	RTO depende del SLA del legado más el tiempo de drenaje de la cola. RPO no aplica (riesgo de disponibilidad, no de integridad).	Plataforma/SRE, con el equipo propietario del legado. La cola se clasifica y procesa por estado (rechazada, incierta, confirmada, 15.2); ninguna operación incierta se reintenta automáticamente si el legado no expone consulta por operationId; tiempo máximo de espera de la cola y de las reservas de fondos respetado.
D. Falla del Registro de Propiedad	Semestral	RTO ~5 min (referencial). RPO ~0 (referencial).	Equipo de plataforma de migración. Auditoría cruzada de tres fuentes completa antes de reabrir enrutamiento; sin reapertura basada en valor cacheado o asumido.
E. Corrupción de datos	Semestral	RTO más largo que el escenario B, por el diagnóstico de causa y alcance. Punto de recuperación: backup más reciente anterior a la corrupción, dentro de la ventana de retención (35 días mínimo).	Plataforma/SRE ejecuta; Seguridad diagnostica la causa; Cumplimiento valida el alcance regulatorio. Tiempo de detección con datos de prueba marcados; backup restaurado sin la causa de la corrupción; el replay excluye el evento que la originó; alcance de restauración acotado.

- **Ejecución:** cada simulacro corre en un entorno que replica producción o mediante fault injection controlado en producción, con ventana anunciada al equipo de Cumplimiento y a los propietarios de cada dominio afectado (14.4).
- **Ajuste cuando el objetivo no se cumple:** si el RTO o el RPO medido no cumple el objetivo declarado en la tabla de componentes de este capítulo, el resultado se documenta como hallazgo abierto y se ajusta la arquitectura, o

bien se corrige el objetivo declarado para que refleje lo que el diseño realmente sostiene. No se deja como una discrepancia implícita entre lo documentado y lo real. Corregir el objetivo declarado requiere aceptación explícita de negocio y Cumplimiento; no es un ajuste editorial unilateral del equipo técnico.

- **Registro:** cada simulacro genera un informe con fecha, escenario ejecutado, RTO/RPO medido frente al objetivo, hallazgos y acciones correctivas con responsable y fecha, mantenido por el equipo de Plataforma/SRE.

16. Plan de Migración Gradual (Strangler Fig)

- **Fase 0 (Preparación):** desplegar el API Gateway/BFF delante del Core Tradicional sin migrar nada; todo el tráfico sigue al legado, pero ya bajo control de enrutamiento centralizado.
- **Fase 1 (Digital-first para lo nuevo):** todo producto nuevo se origina directo en el Core Digital, según la regla de corte ya definida (sección 7.1).
- **Fase 2 (Migración de bajo riesgo):** mover un segmento acotado (p. ej., cuentas de ahorro simples sin productos vinculados) del legado al core digital, validando el patrón end-to-end (CDC, eventos, read model) a pequeña escala.
- **Fase 3 (Migración progresiva por dominio):** repetir para cuentas corrientes y luego préstamos, priorizando por menor complejidad/interdependencia.
- **Fase 4 (Retiro del legado):** cuando el porcentaje de productos activos en el legado caiga bajo un umbral definido (p. ej., <5%), evaluar el retiro completo o mantenerlo como archivo de solo lectura para cumplimiento histórico.

16.1 Migración de una cuenta (Fases 2–3): procedimiento con corte controlado

Mover una cuenta del legado al core digital no es una operación instantánea ni reversible por switch de ruteo; requiere una ventana de corte controlada:

- **Congelamiento de escritura:** la herramienta de migración impide que el Core Tradicional inicie nuevas operaciones sobre esa cuenta (ventana corta, comunicada): no basta con dejar de enrutarle tráfico; el propio core deja de aceptar operaciones nuevas aunque alguien se las envíe directamente. Durante esta etapa, solo se permiten las acciones controladas necesarias para completar o compensar las operaciones que ya estaban en curso (p. ej., un pago en estado pendiente); nunca quedan a medio proceso. Una vez resueltas, se confirma el bloqueo completo de escritura del Core Tradicional para esa cuenta, antes de continuar al siguiente paso.
- **Snapshot y transferencia:** se copia el estado (saldo, historial, productos vinculados) del core tradicional al core digital.
- **Verificación de paridad:** un job compara saldo y últimos movimientos en ambos cores antes de continuar.
- **Corte de propiedad:** la herramienta de migración registra el inicio de esta acción en su registro durable de migración (identificador de migración, producto, origen, destino, versión) y ejecuta, como operaciones separadas (no como una transacción atómica conjunta, sección 7.1), la actualización del Registro de Propiedad de Cuentas y la activación local de "habilitado para escritura" en el Core Digital para la nueva versión. Cada una de las dos acciones se confirma y se anota individualmente en el registro durable de migración a medida que se completa. Este registro durable de migración vive en Oracle Database@Azure, deliberadamente separado del Registro de Propiedad de Cuentas en Redis (7.1), para que la auditoría cruzada de tres fuentes del Escenario D (15.2) no dependa de un único almacén ante una falla; se retiene mientras el corte esté en curso y por un período posterior a su cierre, alineado con la política de auditoría regulatoria.
- **Reapertura de escritura:** la cuenta vuelve a aceptar transacciones, ahora exclusivamente en el Core Digital, que las acepta porque su condición local ya quedó habilitada en el paso anterior.
- **Modo sombra (shadow mode) posterior:** durante un período de observación, las transacciones que habría procesado el legado se simulan (sin efecto real) y se comparan contra el resultado real del core digital, para detectar discrepancias antes de dar la migración por definitiva.

Recuperación de un corte interrumpido: si el proceso se detiene durante o después del paso 4 (p. ej., por una caída de la herramienta de migración), la recuperación no presupone un estado binario predefinido: como el registro central y la habilitación local del Core Digital son acciones separadas (sección 7.1), pudo completarse una sin la otra. La herramienta de migración usa su registro durable (qué acciones quedaron confirmadas) junto con el estado real de las condiciones locales de ambos cores y del registro central para determinar exactamente qué falta: si el registro central se actualizó pero el Core Digital no llegó a habilitarse, completa esa habilitación; si ocurrió al revés, corrige la discrepancia. La escritura (paso 5) solo se reabre cuando el registro central, la habilitación local del Core Digital y el registro durable de migración son coherentes entre sí; nunca por inferencia de un estado "seguro por diseño" asumido de antemano.

16.2 Rollback: qué significa realmente volver atrás

Un rollback vía feature flag en el API Gateway ("volver a enrutar al legado") NO revierte el estado: si el core digital ya procesó movimientos sobre la cuenta migrada, el legado no tiene esos movimientos ni el saldo actualizado. Redirigir el tráfico sin más dejaría al cliente con dos historiales divergentes.

Por eso el rollback real tiene dos escenarios distintos:

- **Rollback antes de confirmar el paso 4 (corte de propiedad) de la sección 16.1:** es seguro y de bajo costo en la mayoría de los casos, pero no debe asumirse que el registro de propiedad permanece intacto solo porque el paso 4 no se dio por completado: al ser dos acciones separadas (actualización del registro central y habilitación local del Core Digital, sección 7.1), es posible que una haya quedado aplicada parcialmente antes de interrumpirse. Este rollback verifica el registro durable de migración y el estado real de ambos cores, y revierte cualquier actualización parcial encontrada, antes de dar la migración por abortada; no se aborta a ciegas asumiendo que nada cambió.
- **Rollback después del corte de propiedad (la cuenta ya opera en el core digital y tiene movimientos propios):** no es un simple cambio de ruteo, sino una migración inversa con el mismo procedimiento de la sección 16.1 pero en sentido contrario, que incluye:
 - Congelar escritura en el core digital para esa cuenta.
 - Reconciliar: calcular la diferencia de movimientos ocurridos en el core digital desde el corte, y aplicarlos como ajustes contables trazables en el core tradicional (no un simple copy-paste de saldo final).
 - Tratar operaciones en curso (pagos en estado pendiente/en curso dentro del esquema) como casos explícitos: se completan en el core digital antes de revertir, o se compensan formalmente (nunca se abandonan a medio proceso).
 - Actualizar el Registro de Propiedad de Cuentas de vuelta al core tradicional, de forma atómica, solo después de reconciliar.
 - Notificar al cliente si hubo algún ajuste visible en su historial.

Condición para permitir el rollback post-corte: solo se autoriza dentro de una ventana de contingencia definida (p. ej., 24–72 horas desde el corte) y mientras el volumen de movimientos posteriores sea reconciliable de forma automática; superado ese umbral, la corrección se maneja como incidente operativo con intervención manual, no como rollback de plataforma.

Mitigación adicional en cada fase: shadow mode antes de cortar tráfico real, comparación de resultados entre legado y digital durante el período de observación, y feature flag en el API Gateway que solo es "instantáneo" en el sentido correcto descrito arriba. En ninguna fase ambos cores son propietarios simultáneos del mismo producto.

17. Observabilidad y Trazabilidad

La Plataforma de Observabilidad es un sistema externo (COTS/SaaS o stack propio tipo OpenTelemetry + backend de métricas/logs/trazas), representada explícitamente en los diagramas de contexto (8.1) y de contenedores (8.2): todos los contenedores le envían telemetría (OTLP); el diagrama de contenedores muestra dos relaciones representativas (API Gateway y Event Bus) para no saturarlo, pero la instrumentación aplica por igual a Core Digital, Payment Hub, BFF y el resto.

- **Distributed tracing:** correlation ID de extremo a extremo (Cliente → BFF → Gateway → dominio → externos), propagado también en los eventos, permitiendo reconstruir el camino completo de una transacción.
- **Métricas:** cada contenedor expone latencia, tasa de error y saturación hacia la Plataforma de Observabilidad.
- **Logging estructurado:** formato JSON con el mismo correlation ID, sin PII en texto plano (alineado a minimización de datos).
- **Trazabilidad de negocio:** cada evento de dominio (pago confirmado, fraude detectado) queda auditado con su ID de correlación, sirviendo tanto a observabilidad técnica como a auditoría de cumplimiento.
- **Alertas basadas en SLO: ejemplo (Payment Hub):** p99 de latencia < 2 s, tasa de error < 0.1%. Este SLO mide la aceptación, validación e inicio de ejecución dentro de Payment Hub (hasta que la orden se entrega al riel de pago); no la liquidación externa, que depende del riel/red de pago correspondiente y tiene su propio SLA, típicamente mayor (p. ej., liquidación ACH puede tardar hasta el día hábil siguiente).

17.1 Responsables y procedimiento ante incidentes

- **Detección:** el equipo de Seguridad/SRE monitorea las alertas de la Plataforma de Observabilidad; un SLO incumplido o un patrón anómalo (p. ej., pico de fallos de autenticación, screening de fraude con latencia degradada) dispara la alerta inicial.
- **Clasificación y contención:** Seguridad/SRE clasifica severidad y ejecuta contención técnica inmediata (aislar el componente afectado, activar circuit breakers, si aplica).
- **Escalamiento a Cumplimiento:** si hay indicio de exposición de datos personales, se escala de inmediato al equipo de Cumplimiento/DPO, quien evalúa si corresponde notificar a la Superintendencia de Protección de Datos Personales y a los titulares, y en qué plazo, conforme a la LOPDP (sección 12).
- **Reconstrucción del alcance:** se apoya en el logging estructurado con correlation ID: permite acotar qué cuentas, qué operaciones y qué ventana de tiempo estuvieron afectadas, insumo necesario tanto para la notificación regulatoria como para la comunicación a los titulares.
- **Cierre y lecciones aprendidas:** todo incidente con impacto en datos personales o en disponibilidad de pagos genera un post-mortem con acciones de seguimiento, propietario y fecha, gestionado por el mismo equipo que lo contuvo.

18. Principios de Diseño Aplicados

- **SRP / responsabilidades cohesionadas:** cada contexto mantiene responsabilidades y reglas de negocio cohesionadas, con propiedad explícita de sus datos y contratos definidos para interactuar con otros contextos (sección 6–7): no se trata de poseer un único tipo de dato (el propio capítulo 6 muestra dominios con varias entidades, p. ej. Pagos), sino de que esas entidades resuelven un mismo ámbito del negocio.
- **Bajo acoplamiento:** comunicación por eventos entre dominios; el API Gateway no acopla consumidores a la topología interna.
- **Segregación de interfaces:** contratos de API específicos por consumidor (minimización de datos, sección 12).
- **Inversión de dependencias:** los dominios Core (Riesgo, Fraude, Pagos) exponen contratos estables; los detalles de implementación (vendor) quedan detrás de esos contratos.
- **Testabilidad y evolución independiente:** cada contenedor se despliega y versiona de forma independiente, sin bases de datos compartidas que acoplen releases.
- **Sobre la clasificación "Core" (capítulo 6):** expresa diferenciación estratégica (dónde el banco compite con su propia política o modelo), no solo criticidad operativa: que Riesgo, Fraude o Pagos participen en el camino de una transacción no basta por sí solo para justificarlos como Core; esa clasificación queda como hipótesis a validar contra la estrategia real del banco, no como una equivalencia automática entre dominio, bounded context y microservicio.
- **Anti-patronos evitados explícitamente:** diagramas genéricos, microservicios sin responsabilidad clara, ownership ambiguo de datos, bases de datos compartidas, cadenas síncronas excesivas, lógica de negocio en el API Gateway, referencias superficiales a BIAN, y controles de seguridad listados sin ubicación clara en la arquitectura.

19. Anexos

19.1 Tabla de Controles de Seguridad

Control	Capa donde aplica	Mecanismo
Autenticación fuerte (SCA/MFA)	Cliente → BFF / API Gateway	OAuth2/OIDC + MFA en operaciones sensibles
Autorización por consentimiento	API Gateway (Open Finance)	Scopes FAPI, validación de consentimiento vigente
Cifrado en tránsito	Todas las integraciones	TLS 1.2+, mTLS interno entre contenedores
Cifrado en reposo	Todos los sistemas de registro	AES-256
Pseudonimización	Event Bus, Vista Agregada	Tokenización de identificadores
Prevención de fraude en tiempo real	API Gateway, Payment Hub → Motor de Fraude	Screening síncrono de baja latencia
Verificación de sanciones/PEP	Payment Hub → Cumplimiento	Sanction Screening antes de ejecutar el pago

Control	Capa donde aplica	Mecanismo
Auditoría de acceso a datos personales	API Gateway, llamadas dominio-a-dominio y consumidores de eventos (ver 12.2)	Logging estructurado con finalidad y consentimiento asociado, correlacionado por el mismo correlation ID en todos los puntos de acceso
Segregación de credenciales de servicio	Todos los contenedores internos	Service accounts individuales + mTLS
Retención y purgado	Cada sistema de registro, y proyecciones/copias derivadas (Vista Agregada, Historial de Interacción en Observabilidad; ver 12.2)	Políticas de retención por dominio, purgado automático; no exentas por ser de solo lectura o vivir en una plataforma primariamente técnica

19.2 Ejemplos de APIs (resumen)

- POST /v1/payments: Payment Hub
- GET /v1/accounts/{id}/balance: Core Digital
- POST /v1/risk/credit-assessment: Motor de Riesgo
- POST /v1/fraud/screen: Motor de Fraude
- POST /openfinance/v1/consents: Plataforma IAM

19.3 Ejemplos de Eventos (resumen)

- account.opened.v1
- payment.initiated.v1
- payment.confirmed.v1
- payment.failed.v1
- fraud.case.created.v1
- legacy.party_reference.updated.v1 (CDC)

20. Conclusiones

La arquitectura propuesta resuelve la modernización bancaria sin forzar una migración disruptiva: el core tradicional permanece como sistema externo integrado mientras el core digital absorbe gradualmente los productos nuevos y migrados, bajo una regla de propiedad de datos única en todo momento. El diseño evita de forma explícita los anti-patrones señalados en el ejercicio (diagramas genéricos, ownership ambiguo, bases de datos compartidas, lógica de negocio en el Gateway y controles de seguridad sin ubicación clara) y sostiene principios SOLID y de Clean Architecture a nivel de dominios y contenedores.

El modelo queda trazable de punta a punta: flujos de valor → capacidades → dominios (validados con BIAN) → sistemas de registro → contenedores C4, lo que permite justificar cada decisión de diseño con su origen de negocio, y no al revés.